

Abschlussprüfung Sommer 2024

Ausbildungsberuf: Fachinformatiker für Anwendungsentwicklung

Dokumentation zur Betrieblichen Projektarbeit

API-Entwicklung in Python mit Flask Swagger-restx und JWT für einen sicheren Datenaustausch via Odoo's XML-RPC

SQLite verwaltet Nutzerrechte, wobei der Zugang je nach Rolle variiert. Für die verschiedenen Endpunkte existieren jeweils unterschiedliche Funktionalitäten.

Abgabedatum: Erfurt, den 24.04.2024

Prüfungsbewerber:

Ferxxxxxx xxxxxxxxxxxx

Adresse

ort plz

Prüflingsnummer: xxx xxxx



Ausbildungsbetrieb:

Informationsverarbeitung und angewandte Datentechnik GmbH

Maximilian-Welsch-Straße 2a

99084 Erfurt

Inhaltsverzeichnis

Anhangsverzeichnis	II
Abbildungsverzeichnis	III
Tabellenverzeichnis	III
Abkürzungsverzeichnis	III
1 Einleitung	1
1.1 Projektbeschreibung	1
1.2 Problemstellung	1
1.2.1 Kontext des Problems	1
1.2.2 Auswirkungen des Problems	2
1.2.3 Betroffene	2
1.3 Projektziel	2
1.4 Projektbegründung	3
1.5 Projektabgrenzung	3
2 Projektplanung	3
2.1 Projektphasen	3
2.2 Ressourcenplanung	4
2.3 Entwicklungsprozess	4
3 Analysephase	5
3.1 Ist-Analyse	5
3.2 Wirtschaftlichkeitsanalyse	5
3.2.1 Make or Buy-Entscheidung	5
3.2.2 Projektkosten	6
3.2.3 Amortisationsdauer	6
3.3 Lastenheft	7
4 Entwurfsphase	8
4.1 Architekturentwurf	8
4.1.1 App-Initialisierung	8
4.1.2 Sicherheitskonzept	9
4.1.3 Nutzerverwaltung	9
4.1.4 Containerisierung	10
4.1.5 Schnittstellen	10
4.2 Zielplattform	11
4.3 Entwurf der Benutzeroberfläche (UI-Design)	11
4.4 Pflichtenheft	11
5 Implementierungsphase	12
5.1 Erstellung des Projektes	12
5.2 Initialisierung und Start des Projekts (run.py):	13
5.3 Hauptklasse und Systemkonfiguration (__init__.py):	13
5.4 Implementierung der Geschäftslogik	13
5.5 Durchführung Einheit(Unittest), Integration(Pytest) und Systemtest	14
5.6 Fehlerbehandlung und Protokollierung	14
5.6.1 Fehlerbehandlung	14
5.6.2 Protokollierung	14
5.7 Abnahme und Qualitätssicherung	14
6 Documentation	15
7 Fazit	15

7.1 Soll-/IST-Vergleich	15
Literaturverzeichnis	16
Anhang A Anhangsverzeichnis	i
A.1 Detaillierte Zeitplanung	i
A.2 Verwendete Ressourcen	ii
A.3 Lastenheft	iii
A.4 Diagram-Amortisationsdauer	iv
A.5 Vorgehensweise	iv
A.6 Aktivitäts-Diagramm	v
A.7 Klassen-Diagramm	vi
A.8 Pflichtenheft (Auszug)	vii
A.9 Auswahl der Programmiersprache Python	viii
A.10 Ablaufzeitplan	ix
7.11 Nutzer-Anleitung	x
Anhang B Abbildungsverzeichnis	I
B.1 Benutzeroberfläche	I
B.2 Admin-Haupt-Userverwaltung-Panel	II
B.3 Projekt Startpunkt	III
B.4 Benutzer Anmeldung Prozess auszuge	III
B.5 Haupt-Klassen-übersicht	IV
B.6 Datenbankstruktur	V
B.7 API und Database Migration	VI
B.8 Odoo Endpunkte und Ressourcen	VII
B.9 Tests und Modulare struktur des Projektes	VIII
B.10 Klassenhierarchie	IX
B.11 SQLite Grafischeoberfläche	IX
B.12 Response Header	IX
Anhang C Anlage IHK	X
C.1 IHK Anlage 1	X
C.2 IHK Anlage 2	XI
C.3 IHK Anlage 3	XII
C.4 IHK Anlage 4	XIII

Abbildungsverzeichnis

1	Zeit (Monaten)	iv
2	Vorgehensmodell	iv
3	Aktivitäts-Diagramm	v
4	Klassendiagramm	vi
5	Ablaufzeitplan des Projekts.	ix
6	Nutzer-Anleitung	x
7	Benutzeroberfläche(GUI)	I
8	CRUD-Funktionalitäten (Create- Read-Update-Delete)	II
9	Startpunkt des Projekts mit benutzerfreundlicher Schnittstelle	III
10	Projekt Auszuge-bevor und nach login	III
11	Hauptstruktur des Quell Code	IV
12	User Database Struktur	V
13	DB Migration Und API Einrichtung)	VI
14	Fehler-Protokolierung und Quell Code-Dokumentation	VII
15	Test-Durchführung	VIII
16	Klassenhierarchie	IX
17	SQLite-GUI-übersicht	IX
18	Abgesicherte Header-Response	IX
19	IHK Anlage 1	X
20	IHK Anlage 2	XI
21	IHK Anlage 3	XII
22	IHK Anlage 4	XIII

Tabellenverzeichnis

1	Grobe Zeitplanung	4
2	Stundensätze	6
3	Kostenaufstellung	6
4	Zeitersparnisse pro Monate	7
6	Soll-/Ist-Vergleich der Projektphasen	15
8	Detaillierte Zeitplanung	i
9	Vergleich zu anderen Programmiersprachen	viii

Abkürzungsverzeichnis

API	Application Programming Interface
GUI	(Graphical User Interface) Grafische Benutzeroberfläche
IAD	Informationsverarbeitung und angewandte Datentechnik
DB	Database
Odoo	On-Demand-Open-Object
SQL	Structured Query Language
JWT	Json Web Token
ERP	Enterprice Ressource Planing (Unternehmensressourcenplanung)
OOP	Objekt Orientierte Programmierung
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ERM	Entity-Relationship-Mode
JSON	JavaScript Object Notation
XML	Extended Markup Language

1 Einleitung

In der folgenden Dokumentation wird das Abschlussprojekt des Autors im Rahmen seiner Prüfung zum Fachinformatiker für Anwendungsentwicklung dargelegt. Durchgeführt wurde dieses bei der IAD GmbH in Erfurt, einem Unternehmen, das seit seiner Gründung 1983 in Marburg und der Eröffnung erster Niederlassungen in Thüringen 1990, darunter Nordhausen, Erfurt und Jena, stetig gewachsen ist.

Heute ist die IAD GmbH anerkannter Partner von namhaften IT-Unternehmen wie Microsoft, Sun Microsystems, IBM, Oracle und anderen, und beschäftigt rund 80 Festangestellte, sowie ebenso viele Freiberufler. Das Expertenteam der IAD GmbH, spezialisiert auf Beratung und Schulung, steht vor der Herausforderung, Schulungen effizient online über Microsoft Teams zu organisieren, inklusive der Erfassung von An- und Abwesenheiten der Teilnehmenden. Um diese Daten direkt in das Odoo-ERP-System einzuspeisen und zu verwalten, wurde der Autor beauftragt, diese Lücke durch die Entwicklung einer Webanwendung zu füllen.

1.1 Projektbeschreibung

Die Hauptaufgabe des Projekts ist die Schaffung einer sicheren Schnittstelle für den Datenaustausch zwischen externen Systemen und Odoo, um Datenintegrität zu sichern und den Zugang zu sensiblen Informationen streng zu überwachen. Sie nutzt Flask-RESTX für die Erstellung einer RESTful API in Python, die eine automatische Dokumentation, Datenvalidierung und Routing unterstützt. Als primäres ERP-System ermöglicht Odoo durch die API-Integration effiziente Geschäftslogik und Datenmanagement. Die Sicherheit wird durch den Einsatz von SQLite zur Verwaltung von Benutzerrechten gewährleistet, was den Zugriff auf die API streng limitiert und somit einen zuverlässigen Schutz sensibler Daten bietet.

1.2 Problemstellung

Eingeschränkter Zugriff auf Odoo: Der Ist-Zustand zeigt, dass der Zugriff auf Odoo aus Sicherheits- und Datenschutzgründen nur innerhalb der Domäne möglich ist. Dies schränkt die Flexibilität und Erreichbarkeit des Systems für externe Anwendungen und Nutzer, die von außerhalb der Domäne auf das System zugreifen müssen, stark ein.

Fehlende digitale Anwesenheitserfassung: Aktuell besteht die Vorgabe für Dozenten, das Klassenbuch und die Anwesenheitserfassung zu führen, was derzeit nicht digital unterstützt wird. Dies verursacht einen erhöhten administrativen Aufwand und kann zu Ungenauigkeiten in der Datenerfassung führen.

Unzureichende Integration für das Klassenbuchprogramm: Es gibt keine vorhandene Schnittstelle, die es dem in Entwicklung befindlichen Klassenbuchprogramm für Teams ermöglicht, effizient auf die benötigten Daten aus Odoo zuzugreifen. Dies behindert die digitale Transformation der Anwesenheitserfassung und des Klassenbuchmanagements.

Mangel an sicherer und kontrollierter Datenzugriffsmöglichkeit: Für externe Anwendungen besteht keine sichere Methode, um auf die notwendigen Daten aus Odoo zuzugreifen, ohne die Sicherheits- und Datenschutzstandards zu kompromittieren.

Fehlen einer eigenständigen Nutzerverwaltung: Es gibt keine dedizierte Lösung für die Verwaltung der Nutzer, die auf die API zugreifen sollen. Dies ist essentiell, um den Zugriff auf die Daten zu kontrollieren und zu sichern.

1.2.1 Kontext des Problems

Die IAD nutzt das ERP-System Odoo zur Verwaltung von Teilnehmenden und Weiterbildungen über mehrere Standorte hinweg. Der Zugriff auf wichtige Funktionen ist jedoch auf die interne Domäne beschränkt, was die digitale Anwesenheitserfassung und das Klassenbuchmanagement, insbesondere für Online-Weiterbildungen via Microsoft Teams, erschwert. Aktuelle Prozesse erfordern manuelle Eingriffe und sind nicht effizient für die digitale Ära ausgerichtet, was die Notwendigkeit einer sicheren und flexiblen Zugriffslösung hervorhebt.

1.2.2 Auswirkungen des Problems

Potenzielle Fehlerquellen bei der Dateneingabe und eine erhöhte administrative Last für die Dozenten können zu Ungenauigkeiten bei der Teilnehmerverwaltung führen, die wiederum die Qualität der Bildungsangebote beeinträchtigen und zusätzlichen Aufwand für Korrekturen erfordern. Die Einschränkungen behindern zudem die Möglichkeiten, das Bildungsangebot flexibel und standortunabhängig zu gestalten.

1.2.3 Betroffene

Primär betroffen sind die Dozenten, die einen erhöhten Verwaltungsaufwand betreiben müssen, und die Teilnehmenden, deren Lernerfahrung durch ineffiziente Prozesse beeinträchtigt werden. Des Weiteren sind die administrativen Mitarbeiter betroffen, die mit der Datenpflege und -korrektur befasst sind, sowie das IT-Personal, das nach sicheren Lösungen für den Datenzugriff und die Nutzerverwaltung suchen muss.

1.3 Projektziel

Durch die Implementierung der Automatisierung des Klassenbuchmanagements und der Anwesenheitserfassung zielt dieses Projekt darauf ab, zeitaufwendige manuelle Prozesse zu eliminieren. Dies stellt den ersten Schritt dar, um eine signifikante Reduzierung des administrativen Aufwands zu erreichen und ermöglicht es Dozenten, sowie Verwaltungspersonal, sich stärker auf ihre Kernkompetenzen zu fokussieren und die Bildungsqualität zu verbessern. Im Folgenden werden die spezifischen, klar definierten Projektziele detailliert erläutert.

- **Verbesserung der Betriebseffizienz:** Das Hauptziel dieses Projekts ist die Schaffung einer sicheren und effizienten Schnittstelle für den Datenaustausch, um die Integrität und Vertraulichkeit der Daten zu gewährleisten. Durch die Entwicklung dieser API wurde eine Middleware-Lösung realisiert.
- **Erhöhung der Datenintegrität und -sicherheit:** Die sichere API garantiert einen kontrollierten Zugriff auf die Daten in Odoo, minimiert das Risiko von Datenlecks und sorgt für eine hohe Datenqualität. Durch die Implementierung von Best Practices für die Datensicherheit und die Nutzung einer eigenständigen SQL-Datenbank für die Nutzerverwaltung wird das Datenschutzniveau signifikant erhöht.
- **Flexibilität und Skalierbarkeit:** Die Entwicklung der API als Docker-Container ermöglicht eine einfache Integration in die bestehende Infrastruktur und gewährleistet gleichzeitig die Skalierbarkeit des Systems. Die Lösung kann somit problemlos an wachsende Anforderungen und Teilnehmerzahlen angepasst werden.
- **Unterstützung digitaler Bildungsformate:** In Anbetracht der steigenden Bedeutung digitaler Lernformate ermöglicht die API die nahtlose Verwaltung von Online- und Präsenzweiterbildungen. Die digitale Anwesenheitserfassung und das Klassenbuchmanagement unterstützen sowohl synchrone als auch asynchrone Lernformate und tragen damit zur Flexibilisierung des Bildungsangebots bei.
- **Reaktion auf Nutzerbedürfnisse:** Durch die Bereitstellung einer Schnittstelle für das Klassenbuchprogramm in Teams wird auf die spezifischen Anforderungen der Dozenten und Teilnehmenden reagiert. Die nutzerfreundliche Gestaltung fördert die Akzeptanz und die effektive Nutzung des Systems.
- **Wettbewerbsvorteil und Imageförderung:** Die Digitalisierung interner Prozesse und die Implementierung fortschrittlicher Technologielösungen stärken das Image der IAD als moderner und innovativer Bildungsträger. Dies kann einen entscheidenden Wettbewerbsvorteil darstellen und die Attraktivität für potenzielle Teilnehmende und Dozenten erhöhen.
- **Effizienzsteigerung:** Automatisierung des Datenaustausches zwischen Systemen, was zu einer Reduzierung manueller Eingriffe und potentiellen Fehlern führt.

- **Sicherer Datenaustausch:** Gewährleistung, dass keine sensiblen Daten unbefugt zugänglich sind.
- **Erweiterbarkeit:** Die Architektur der API ermöglicht es, zukünftige Anforderungen und Integrationen effizient zu implementieren.

1.4 Projektbegründung

Der Entwickler hat sich aus folgenden Gründen entschieden eine App Mit Flask Swagger-ui zu entwickeln, die auf Flask-RESTful (RestX) basiert, und als Middleware verwendet wird. Die Notwendigkeit für dieses Projekt ergibt sich direkt aus den kritischen Herausforderungen und Einschränkungen, wie zum Beispiel:

- Eingeschränkter Zugriff auf Odoo
- Fehlende digitale Anwesenheitserfassung
- Unzureichende Integration für das Klassenbuchprogramm

Daneben wurden in dem Abschnitt [1.2: Problemstellung Seit 1](#) weitere Hauptgründe identifiziert. Diese Probleme beeinträchtigen die Effizienz, Sicherheit und Flexibilität der IT-Infrastruktur und -Prozesse erheblich und stehen einer modernen, digitalisierten Bildungseinrichtung entgegen.

1.5 Projektabgrenzung

Ausschlüsse (Was das Projekt nicht beinhaltet):

- **Überarbeitung des Odoo-ERP-Systems:** Das Projekt umfasst nicht die Modifikation oder Erweiterung der bestehenden Funktionen von Odoo außerhalb der für die API benötigten Schnittstellen.
- **Entwicklung der Klassenbuch-App:** Während die API Schnittstellen für die Klassenbuch-App bereitgestellt werden, ist die Entwicklung dieser App selbst nicht Teil dieses Projekts.
- **Hardware-Beschaffung:** Die Anschaffung von zusätzlicher Hardware oder Infrastruktur für die Implementierung oder den Betrieb der API fällt nicht in den Projektumfang.
- **Containerisierung:** Die Einbettung der API-Anwendung in einen Docker-Container, sowie jegliche damit verbundene Bereitstellungs- und Skalierungsmaßnahmen, sind nicht Teil dieses Projekts. Diese Aufgaben werden separat von einem anderen Team oder einer anderen Abteilung übernommen.

2 Projektplanung

In der Phase der Projektplanung ist es entscheidend, einen umfassenden Plan zu entwickeln, der sowohl den Zeitrahmen, als auch die erforderlichen Ressourcen detailliert abbildet. Dabei wird auch der sequenzielle Ablauf der Projektschritte festgelegt, um eine effiziente und zielgerichtete Umsetzung zu gewährleisten.

2.1 Projektphasen

Für die Durchführung des Projekts sind insgesamt 80 Stunden vorgesehen. Diese Zeit ist im Vorfeld des Projektstarts strategisch auf verschiedene Phasen der Softwareentwicklung aufgeteilt worden. Eine vorläufige Aufschlüsselung der Zeitverteilung, sowie eine Auflistung der Hauptphasen, sind in Tabelle [A.1: Grobe Zeitplanung](#) (seit 4) dargestellt. Darüber hinaus besteht die Möglichkeit, jede Hauptphase in feinere Unterphasen zu gliedern, um eine detailliertere Planung und Nachverfolgung zu ermöglichen. Eine eingehendere Darstellung dieser Unterteilung ist im Anhang [A.2: Detaillierte Zeitplanung](#) auf Seite (i) zu finden.

Projektphase	Geplante Zeit
Analyse	10 h
Entwurf	10 h
Implementierung	35 h
Abnahme und Einführung	5 h
Dokumentation	20 h
Gesamt	80 h

Tabelle 1: Grobe Zeitplanung

2.2 Ressourcenplanung

Im Anhang [A.2: Verwendete Ressourcen](#) Seit(i) finden Sie eine detaillierte Auflistung aller Ressourcen, die während des Projekts zum Einsatz kamen. Dies umfasst Hard- und Software, sowie das beteiligte Personal. Bei der Software-Auswahl wurde ein Fokus auf kostenfreie Optionen, wie Open-Source-Lösungen, gelegt oder auf Programme mit bereits verfügbaren Lizenzen. Diese Strategie diente dazu, die Projektkosten so niedrig wie möglich zu halten.

2.3 Entwicklungsprozess

Aufgrund der begrenzten Zeit und der präzisen Definition der Anforderungen im Lastenheft hat sich der Entwickler entschieden, den Entwicklungsprozess des Projekts gemäß dem V-Modell durchzuführen. Dieses Modell, bekannt für seine strukturierte Herangehensweise und die klare Zuordnung von Testphasen zu Entwicklungsstufen, bietet einen optimalen Rahmen, um Effizienz und Qualität innerhalb enger Zeitvorgaben zu gewährleisten, eine Übersicht des V-Modell [Abbildung: Vorgehensmodell](#) auf der Seite [\(v\)](#) steht zur Verfügung.

Grundlagen des gewählten Entwicklungsprozesses:

Das V-Modell wurde ausgewählt, da es eine systematische Abfolge von Schritten bietet, die eng mit den vordefinierten Anforderungen im Lastenheft verknüpft sind. Durch die frühe Festlegung der Reihenfolge der Entwicklungsschritte im Anhang des [A.3: Lastenheft](#) seit (iii) konnte der Entwickler den Entwicklungsprozess präzise planen und dabei sicherstellen, dass alle erforderlichen Komponenten und Funktionen berücksichtigt werden.

Durchführung gemäß dem V-Modell:

Der Prozess begann mit der detaillierten Analyse der Anforderungen aus dem Lastenheft, um ein tiefes Verständnis für die Projektziele und die erforderlichen Funktionalitäten zu entwickeln. Aufbauend darauf erfolgte der Entwurf der Systemarchitektur und der einzelnen Module, wobei besonderes Augenmerk auf die Modularität und Integrationsschnittstellen gelegt wurde.

Mit der Implementierungsphase startete die konkrete Umsetzung der Software, wobei jedes Modul gemäß den festgelegten Spezifikationen entwickelt und anschließend einem Modultest unterzogen wurde. Diese parallele Entwicklung und Testung ermöglichte eine effiziente Fehlererkennung und -behebung. Nach der Fertigstellung der Module folgte die Integrationsphase, in der die einzelnen Komponenten zu einem Gesamtsystem zusammengeführt wurden. Entsprechende Systemtests validierten die Interoperabilität der Module und die Erfüllung der Gesamtanforderungen. Abschließend wurden im Rahmen der Abnahmetests die Funktionalitäten des Systems gegenüber den im Lastenheft festgelegten Kriterien geprüft, um die Übereinstimmung mit den ursprünglichen Projektzielen sicherzustellen. Die erfolgreiche Durchführung dieser Tests markierte den Übergang in die Wartungs- und Supportphase, während das Team kontinuierlich für die Optimierung und Anpassung des Systems zur Verfügung stand.

3 Analysephase

3.1 Ist-Analyse

Die IAD setzt das ERP-System Odoo für die Verwaltung ihrer Weiterbildungen an verschiedenen Standorten ein. Der Zugang zu Odoo ist ausschließlich innerhalb der internen Domäne möglich, was die Umsetzung digitaler Lösungen für das Klassenbuchmanagement und die Anwesenheitserfassung, besonders bei Online-Weiterbildungen über Microsoft Teams, behindert. Die aktuellen, manuell geprägten Verfahren sind ineffizient und nicht zeitgemäß. Es besteht ein dringender Bedarf für eine Lösung, die einen sicheren und flexiblen Zugriff auf Odoo auch von außerhalb der Domäne erlaubt, um die digitale Transformation der IAD voranzutreiben. Zusätzlich sind folgende Punkte zu berücksichtigen:

Überarbeitung des Odoo-ERP-Systems:

- **Verwaltung über ERP-System Odoo:** Sicher und zuverlässig, aber nur innerhalb der Domäne zugänglich.
- **Durchführung von Weiterbildungen:** Flexibel, sowohl vor Ort als auch online über Microsoft Teams.
- **Anforderung zur Digitalisierung:** Es besteht ein dringendes Bedürfnis nach der Digitalisierung des Klassenbuchs und der Erfassung der Anwesenheit, um die Verwaltung zu vereinfachen und die Effizienz zu steigern.
- **Fehlende Integration:** Aktuell ist keine direkte Integration externer Anwendungen mit Odoo möglich, besonders von Anwendungen außerhalb der Domäne.

Erforderliche Maßnahme:

Derzeit wird die Klassenbuchführung manuell auf Papier durchgeführt. Für jede Klasse wird eine Anwesenheitsliste erstellt, physisch ausgefüllt, anschließend gescannt und dann an den Dozenten gesendet. Dieser Prozess ist zeitaufwändig und fehleranfällig, weshalb die Entwicklung einer API, mit sicheren und eingeschränkten Zugriff auf Odoo (außerhalb der Domäne) notwendig ist, um die Integration der Klassenbuch-App zu erleichtern und den digitalen Transformationsprozess innerhalb der IAD voranzutreiben.

3.2 Wirtschaftlichkeitsanalyse

Aufgrund der in den Abschnitten [1.2: Problemstellung](#) seit (1) und [1.4: Projektbegründung](#) seit(3) dargelegten Probleme des momentanen Übertragungswertverarbeitungsprozesses und den Einschränkungen im Zugriff auf Odoo, ist die Umsetzung des Projektes zwingend notwendig. Die folgenden Abschnitte untersuchen, ob die Realisierung des Projekts auch aus wirtschaftlicher Perspektive gerechtfertigt ist.

3.2.1 Make or Buy-Entscheidung

Diese Entscheidung basiert auf einer eingehenden Analyse der Verfügbarkeit von marktfertigen Lösungen im Vergleich zu den Anforderungen des Projekts. Die einzigartigen Anforderungen an Sicherheit, Datenschutz und die spezifische Integration in das bestehende Odoo-System machen eine maßgeschneiderte Entwicklung erforderlich. Eine Marktanalyse hat gezeigt, dass keine vorhandene Lösung die spezifischen Bedürfnisse hinsichtlich Flexibilität, Erreichbarkeit und Sicherheit in vollem Umfang erfüllen kann. Dies begründet die Entscheidung zur Eigenentwicklung der Middleware.

3.2.2 Projektkosten

Der Entwickler hat eine detaillierte Kostenschätzung für die verschiedenen Phasen des Projekts vorgestellt. Dabei werden sowohl die Personalkosten der Entwickler und aller weiteren am Projekt beteiligten Personen berücksichtigt, als auch die Ausgaben für benötigte Ressourcen, die bereits in der Ressourcenplanung Abschnitt [A.2: Verwendete Ressourcen](#) auf der Seite (ii) definiert wurden.

Aufgrund von Datenschutzbestimmungen können die exakten Personalkosten nicht offengelegt werden. Stattdessen basiert die Kostenschätzung auf durchschnittlichen Stundensätzen, die von der Personalabteilung vorgegeben sind. Diese Stundensätze umfassen in der Regel das Bruttoentgelt der Mitarbeiter, sowie Arbeitgeberbeiträge zur Sozialversicherung. Zusätzlich werden die Kosten für die Verwendung der spezifizierten Ressourcen berücksichtigt. Für die Kalkulation wird ein Stundensatz von 40 Euro für reguläre Mitarbeiter angesetzt. Auszubildender werden mit einem Stundensatz von 10 Euro berücksichtigt. Für die Nutzung der erforderlichen Ressourcen wird pauschal ein Betrag von 15 Euro pro Stunde veranschlagt.

Das Projekt ist auf eine Dauer von 80 Stunden ausgelegt. Die Kosten sind in der Kostenaufstellung [Tabelle 3: Kostenaufstellung](#) nach den verschiedenen Aktivitäten des Projekts aufgeschlüsselt und zum Abschluss zusammengefasst, um die Gesamtkosten (Stromkosten, Büromietkosten, Personalkosten, Hard- und Software, usw.) zu ermitteln. Die geschätzten Gesamtkosten belaufen sich auf insgesamt 2.165,00 Euro.

Kosten	Kosten / Stunde
Auszubildender	10 €
Mitarbeiter	40 €
Ressourcenkosten(z.B. Strom usw...)	15 €

Tabelle 2: Stundensätze

Vorgang	Mitarbeiter	Zeit	Personal	Ressourcen	Gesamte
Entwicklungskosten	1 x Auszubildender	80h	800,00 €	1200,00 €	2.000,00 €
Code-Review	1 x Mitarbeiter	2h	80,00 €	30,00 €	110,00 €
Abnahme	1 x Mitarbeiter	1h	40,00 €	15,00 €	55,00 €
Gesamte Projektkosten					2.165,00 €

Tabelle 3: Kostenaufstellung

3.2.3 Amortisationsdauer

Die Amortisationsdauer wird anhand der Einsparungen berechnet, die durch die Implementierung der Middleware erzielt werden. Dies umfasst die Reduzierung des administrativen Aufwands, die Vermeidung von Fehlern durch manuelle Dateneingaben, die Verbesserung der Datensicherheit und den schnelleren Zugriff auf Odoo-Daten. Basierend auf einer konservativen Schätzung der jährlichen Einsparungen wird erwartet, dass die Investitionskosten von einem Jahr vollständig amortisiert sein sollen, um die Wirtschaftlichkeit des Projekts vollständig zu bewerten.

Amortisationsdauer basierend auf Zeiteneinsparung:

Die Effizienzsteigerung durch das Projekt führt zu einer signifikanten Zeitersparnis bei den täglichen Arbeitsabläufen. Die Amortisationsdauer basiert auf einer monatlichen Zeitersparnis von 367,3 Minuten, die durch das Projekt durch Optimierung von Vorbereitungszeiten mittels wöchentlichem Scannen und Verschicken von Klassenbüchern erreicht wird. Eine detaillierte Übersicht dieser [Tabelle 4: Zeitersparnisse pro Monat](#) auf der Seite (7) wird in einer Tabelle dargestellt. Darüber hinaus findet sich unter der Tabelle eine umfassende Kalkulation der Amortisationsdauer. Für eine vertiefte Analyse der Kosteneinsparungen und der daraus resultierenden Amortisationsdauer, verweist der Entwickler auf ein Diagramm im Anhang [A Abbildung 1: Amortisationsdauer](#) auf der

seit (iv) Unter Berücksichtigung der Personalkosten und der Ressourcennutzung führen die Projektkosten von 2,165,00 € zu einer Amortisationsdauer von etwa 6 Monaten, was eine schnelle Rentabilität und somit einen wirtschaftlichen Erfolg für die IAD GmbH signalisiert.

Beschreibung	Anzahl Pro Monat	Anzahl API Server/ Personal	Zeit alt Pro Vorang	Zeit neu Pro Vorang	Einsparung Pro Monat
Dozent-Vorbereitungszeit	4	10	8 min	0.50 min	300 min
Mitarbeiter-Vorbereitungszeit	4	2	8 min	0.50 min	60 min
Vorbereitungszeit/Ladezeit	4	1	2 min	0.16 min	7.33min
Zeiteinsparung/Monate					367.3Min

Tabelle 4: Zeitersparnisse pro Monate

$$4 * 10 * 8 = 320 - 20 = 300Min||, 4 * 2 * 8 = 64 - 4 = 60Min||, 4 * 1 * 2 = 8 - 0.67 \approx 7.33Min \quad (\text{Tipps})$$

$$367.3 \frac{\text{min}}{\text{Monat}} \times \frac{12 \text{ Monate}}{\text{Jahr}} = 4.407.6 \frac{\text{min}}{\text{Jahr}} = 73.46 \frac{\text{h}}{\text{Jahr}} \quad (1)$$

$$73.46 \frac{\text{h}}{\text{Jahr}} \times (40\text{€} + 15\text{€}) = 4.040.3 \frac{\text{€}}{\text{Jahr}} \quad (2)$$

$$\frac{2.165.00\text{€}}{4.040,3 \frac{\text{€}}{\text{Jahr}}} = 0.53 \text{ Jahre} \approx 6.36 \text{ Monaten} \quad (3)$$

Amortisation basierend auf Arbeitsstunden eines Mitarbeiters:

Die Investition in das Projekt beläuft sich auf 2.165.00 €, wobei die Mitarbeiterkosten 40 € pro Stunde und die Ressourcenpauschale 15 € pro Stunde betragen. Unter der Annahme, dass durch das Projekt operative Effizienz erreicht wird, sodass die Kosten für Arbeitszeit und Ressourcenverbrauch gesenkt werden, lässt sich die Amortisation auf die Zeit der Arbeitersparnis herunterbrechen. Theoretisch amortisiert sich das Projekt, wenn der Wert der durch das Projekt eingesparten Arbeitsstunden die Projektkosten deckt, was in diesem Fall nach 39.36 Stunden der Fall wäre:

$$T = \frac{\text{Projektkosten}}{\text{Mitarbeiterkosten pro Stunde} + \text{Ressourcenpauschale pro Stunde}} \quad (1)$$

Setzt man die gegebenen Werte ein, ergibt sich:

$$T = \frac{2.165.00 \text{ €}}{40 \text{ €/h} + 15 \text{ €/h}} \approx 39.36 \text{ h} \quad (2)$$

Diese Berechnung dient als erste Schätzung. Die tatsächliche Amortisationsdauer wird durch die im betrieblichen Alltag realisierten Einsparungen bestimmt.

3.3 Lastenheft

Am Ende der Analysephase wurde gemeinsam mit dem Abteilungs-Geschäftsführer ein Lastenheft erstellt. In dem Lastenheft sind alle zu berücksichtigenden Anforderungen des Auftraggebers an die zu implementierende Anwendung, sortiert nach der Wichtigkeit der Umsetzung, festgehalten. Die Formulierung wurde mit Hilfe der Must-, Should-, Could-, Would- (MoSCoW)-Methode umgesetzt und ist, nach Wichtigkeit priorisiert, dargestellt. Das heißt, dass in jeder Anforderung die Wichtigkeit aufgeschlüsselt nach „muss“, „soll“, „kann“ oder „würde gerne“ festgehalten wird. Anhand dieser Methodik wurde auch die Priorisierung vorgenommen. Das vollständige Auszüge aus dem Lastenheft befindet sich im Anhang [A.3: Lastenheft](#) auf der seit (iii).

4 Entwurfsphase

Das vorliegende Projekt ist eine Flask-RESTx basierende API, die sich mit dem Odoo-ERP-System über eine XML-RPC Schnittstelle verbindet, und eine maßgebliche Bedeutung für den Austausch kritischer Geschäftsdaten zwischen den System hat. Die Implementierung nutzt SQLite zur Verwaltung von Zugriffsrechten, um die Sicherheit der Daten und die Integrität der Anwendungslogik zu gewährleisten.

4.1 Architekturentwurf

Gesamtarchitektur: Der Hauptteil des Projekts besteht aus Flask und [Swagger-UI](#) (seit I) mit [JWT](#), um eine kompatible grafische Oberfläche bereitzustellen. Anschließend wird die Odoo ERP-Datenbank für den Datenaustausch zwischen internen und externen Services verwendet. Um die Daten zwischen internen und externen Quellen sowie zwischen dem Klassenbuchprogramm und Odoo in einer sicheren Umgebung zu verwalten, muss eine App entwickelt werden, als Middleware, die den Anforderungen entspricht.

Für die Sicherheit der Endpunkte muss eine SQLite-Datenbank verwendet werden, um spezifische Rechte für bestimmte Nutzer oder Dozenten bezüglich des App-Zugriffs festzulegen. Schließlich muss die App in einem Docker-Container umgesetzt werden.

4.1.1 App-Initialisierung

Die Anwendung beginnt mit der Erstellung einer Flask-Instanz, die als Herzstück und als zentraler Koordinationspunkt fungiert. Diese Instanz ist verantwortlich für die Initialisierung und Konfiguration der Kernaspekte der Anwendung, einschließlich der Definition von URL-Routen, Anfrage-Handlern und weiteren anwendungsspezifischen Einstellungen.

Modularer Aufbau: Um eine klare Struktur und bessere Wartbarkeit zu gewährleisten, ist die Anwendung in verschiedene Module aufgeteilt. Die Kernkomponenten des Projekts sind in einem Paket organisiert, dass im Wurzelverzeichnis unter dem Namen app bereitgestellt wird. Ein separates Startskript, run.py, befindet sich im Root-Verzeichnis, um den Start der Anwendung zu erleichtern [B.3 Abbildung.9: Projektstart punkt](#). Dies ist besonders nützlich für den Einsatz in einem Docker-Container, da es die Bereitstellung und Ausführung der Anwendung vereinfacht, modulare aufbau des Projekt im [B.9 Abbildung15: Tests und moduls](#) zusehen ist.

Die Hauptklasse MyApplication: Innerhalb der Anwendung wurde eine zentrale Klasse namens MyApplication definiert, die als Hauptklasse dient. Diese Klasse implementiert verschiedene Setup-Funktionen unter Verwendung der objektorientierten Programmierung (OOP), um globalen Variablen und bestimmten Daten Zugriff über verschiedene Klassen, Methoden und Module hinweg zu ermöglichen. Eine Unterklasse namens ClassConfiguration ist speziell für SQL-Konfigurationen vorgesehen und ermöglicht es, die Datenbankkonfiguration separat zu verwalten und bei Bedarf zu erweitern, eine übersicht des Hauptklasse befindet sich im [Abbildung.11: Hauptklasse](#) seit (IV).

Modularisierung der Funktionalitäten: Die Anwendung nutzt Modularisierung intensiv, um ihre Funktionalitäten zu organisieren und zukünftige Erweiterungen zu erleichtern. Darunter fallen folgende Aspekte:

- **Odoo-Integration:** Ein Modul ist für die Verbindung und Interaktion mit der Odoo-ERP-Software zuständig. Dies ermöglicht eine nahtlose Integration von Geschäftsprozessen und Datenmanagement.
- **Datenbankstrukturen:** Die Struktur der Datenbank und alle Datenmodelle sind in einem eigenen Modul definiert. Dies fördert eine klare Trennung von Datenlogik und Anwendungslogik.
- **Odoo Ressourcen:** Weitere Module sind spezifisch für die Verwaltung von Odoo-Ressourcen und Nutzerdaten vorgesehen. Diese Organisation unterstützt eine modulare Entwicklung und Erweiterung der Anwendung.

Routen-Definition und View-Funktionen: Flask verwendet Dekoratoren, um URL-Routen zu definieren und mit Funktionen zu verknüpfen; Dekoratoren werden als View-Funktionen bezeichnet. Diese Funktionen werden aufgerufen, wenn eine HTTP-Anfrage mit der zugeordneten Route übereinstimmt. Innerhalb der View-Funktionen wird die Logik zur Verarbeitung der Anfragen und zum Zurücksenden der Antworten implementiert.

Anfrage-Handling: Flask vereinfacht die Handhabung eingehender HTTP-Anfragen, indem es Objekte bereitstellt, die den Anfragekontext enthalten, wie z.B. request. Entwickler können auf Daten wie Query-Parameter, URL-Parameter, Formulardaten und JSON-Inhalte aus dem Anfragekörper zugreifen.

Antwort-Generierung: Flask ermöglicht die einfache Erstellung von HTTP-Antworten, die entweder aus einfachen Zeichenketten, HTML-Inhalten oder JSON-Daten bestehen können. Durch die Verwendung der jsonify-Hilfsfunktion kann Flask automatisch Python-Diktate in JSON-Antworten umwandeln, was für RESTful-APIs essentiell ist.

Fehlerbehandlung und Middleware: Flask bietet Mechanismen zur Fehlerbehandlung und Unterstützung für Middleware-Komponenten, die es ermöglichen, Code vor oder nach Anfragen auszuführen, was für Authentifizierung, Autorisierung und Logging nützlich ist.

Erleichterung durch Flask: Flask erleichtert die Erstellung von Webservern und die Handhabung von HTTP-Anfragen. Eine eingebaute Entwicklungsumgebung und ein Debugger beschleunigen außerdem das Entwickeln und das Fehler-Identifizieren der Anwendung. Die Erstellung von RESTful APIs durch unterstützende Funktionen wie jsonify vereinfachen die JSON-Serialisierung von Antwortdaten.

4.1.2 Sicherheitskonzept

Authentifizierung und Autorisierung: Für die Registrierung von Nutzern wird ein spezifischer Prozess angewendet, bei dem Nutzer entweder als normale Nutzer oder Super-Nutzer in einer SQLite-Datenbank registriert werden können. Die Passwörter der Nutzer werden unter Verwendung des SHA-256-Algorithmus gehasht, bevor sie in der Datenbank gespeichert werden, um die Sicherheit der Nutzercredentials zu gewährleisten.

Die Anmeldung nutzt HTTP Basic Auth mit JSON Web Tokens (JWT), die Nutzerdaten, einen geheimen Schlüssel und eine Ablaufzeit enthalten, welche nach erfolgreicher Passwortprüfung generiert werden. Diese Tokens sind für weitere Anfragen erforderlich. Zur Sicherheitssteigerung wurden Rate Limiting gegen DoS-Angriffe und sichere Header wie X-Frame-Options gegen Clickjacking eingeführt, und weitere sicherheitsmaßnahmen, eine kleine übersicht aus dem Header response im [Abbildung.18: Response Header](#) ist zuverfügung.

4.1.3 Nutzerverwaltung

Nutzerverwaltung in SQLite: SQLite bietet eine einfache und effiziente Lösung für die Speicherung von Nutzerdaten. Jeder Nutzer wird in der Datenbank mit einer eindeutigen ID, einem Benutzernamen und einem Passwort gespeichert. Die Passwörter der Nutzer werden vor der Speicherung in der Datenbank mit einem Hashing-Algorithmus wie SHA-256 gehasht, um zusätzliche Sicherheit zu bieten und sicherzustellen, dass Passwörter nicht im Klartext gespeichert werden. ([Abbildung.17: SQLite GUI](#)) Übersicht weitere Attributen wie Rollen oder Berechtigungen, die gespeichert sind.

Die Nutzung von SQLite für die Nutzerverwaltung bietet mehrere Vorteile:

- **Leichtgewichtig und Einfach:** SQLite benötigt keine separate Serverprozesse oder -konfigurationen, was die Einrichtung und Wartung vereinfacht.
- **Portabilität:** Da SQLite-Datenbanken als einfache Datei auf dem Dateisystem gespeichert sind, ist es einfach, Datenbanken zwischen verschiedenen Umgebungen oder Systemen zu migrieren.

- **Geringe Betriebskosten:** SQLite ist für kleinere Anwendungen kostenlos und erfordert keine teuren Datenbankmanagement-Systeme.

Authentifizierung der Nutzer: Die Authentifizierung in der API erfolgt durch HTTP Basic Auth und token-basierte Verfahren mit Ablaufzeiten. Benutzer werden über ihren Benutzernamen und Passwort authentifiziert, woraufhin ein zeitlich begrenzter JWT für den Zugriff auf geschützte Ressourcen ausgestellt wird. Diese Methode verbessert die Sicherheit und Effizienz der Authentifizierung.

4.1.4 Containerisierung

Containerisierung ist eine Methode zur effizienten Softwareverteilung, bei der Anwendungen mit ihren Abhängigkeiten und Teilen des Betriebssystems in isolierten Containern verpackt werden. Diese Methode sorgt für Konsistenz und Portabilität über verschiedene Umgebungen hinweg, reduziert Ressourcenverbrauch und erhöht die Startgeschwindigkeit im Vergleich zu traditionellen virtuellen Maschinen. Zudem bietet sie durch Isolation eine erhöhte Sicherheit, sodass ein Problem in einem Container nicht die gesamte Systemumgebung betrifft. Container erleichtern auch die Skalierung und Verwaltung von Anwendungen, besonders mit Orchestrierungswerkzeugen wie Kubernetes, die automatisierte Bereitstellung und Skalierung ermöglichen.

4.1.5 Schnittstellen

Odoo Daten-Austausch:

- **Zweck:** Zur Sicherung der Datenübertragung mit Odoo wurden spezifische Sicherheitsmaßnahmen für jede Route eingeführt. Änderungen via POST erfordern Superuser-Berechtigungen.

Hier ist eine erweiterte Beschreibung der Endpunkte und der zugehörigen Sicherheitsmaßnahmen:

- **Endpunkte:**
 - **GET /odoo/maßnahme/{id}:** Abrufen von Maßnahmedaten mittels ihrer ID. Dabei wird marshal-with für die Formatanpassung innerhalb der Flask-API verwendet. Der Zugriff ist durch Benutzeranmeldung und JWT gesichert und umfasst zusätzliche erweiterte Sicherheitsmaßnahmen.
 - **POST /odoo/add_Data_Odoo:** Hinzufügen neuer Maßnahmen oder Daten in Odoo ist beschränkt auf Superuser oder Benutzer mit erweiterten Rechten. Der Zugriff ist abgesichert durch Superuser-Login und JWT sowie durch erweiterte Sicherheitsmechanismen..
 - **DELETE /odoo/Remove_odoo_data/{Partner.id}:** Löschen unnötiger Daten oder Maßnahmen von Odoo anhand des Partner ID.

Account Management:

- **Zweck:** Verwaltung von Benutzerrollen und Berechtigungen über eine SQLite-Datenbank.
- **Endpoints (CRUD-Funktionen für Benutzerverwaltung):**
 - **POST /accounts:** Hinzufügen neuer Benutzer.
 - **GET /accounts/id{id}:** Aufrufen von Benutzerdaten.
 - **PUT /accounts/id{id}:** Aktualisieren von Benutzerdaten.
 - **DELETE /accounts/id{id}:** Löschen von Benutzern.
- **Zugriffskontrolle:** Nur Superuser haben die Berechtigung, Benutzer zu verwalten. Die Rolle des Superusers wird vom Leiter des IAD, Herr Neuhausen, verwaltet.

4.2 Zielplattform

Die Anwendung ist für den Einsatz auf folgenden Plattformen konzipiert:

Entwicklungsplattform: Die Software wurde unter Windows 10 Enterprise entwickelt, mit einem Fokus auf Stabilität und Leistung in professionellen Umgebungen.

Testumgebung: Für Testzwecke wurde Hyper-V zur Simulation verschiedener Betriebsumgebungen genutzt. Dies gewährleistet, dass die Anwendung unter diversen Bedingungen zuverlässig funktioniert.

Serverumgebung: Der Backend-Service basiert auf Python 3.10 und verwendet Flask-Swagger-UI mit Flask-RestX zur Bereitstellung einer interaktiven API-Dokumentation. Diese Kombination wurde gewählt, um eine robuste, leicht zugängliche und gut dokumentierte Backend-Infrastruktur zu schaffen.

Webbrowser: Die Benutzeroberfläche ist webbasiert und optimiert für die neuesten Versionen von Chrome, Firefox, Safari und Edge, um eine hohe Benutzerfreundlichkeit und Performance zu gewährleisten.

Diese Zielplattformbeschreibung gewährleistet, dass alle Beteiligten verstehen, auf welchen Systemen die Anwendung entwickelt, getestet und eingesetzt wird. Sie umfasst die Entwicklungsumgebung und die Nutzererfahrung, um Kompatibilität und Zugänglichkeit sicherzustellen.

4.3 Entwurf der Benutzeroberfläche (UI-Design)

Entsprechend den Abschnitt [A.3: Lastenheft](#) Anforderungen zur Integration von Flask-Swagger wurde die Benutzeroberfläche benutzerfreundlich und zugänglich entworfen. Flask-Swagger ermöglicht die Erstellung einer interaktiven API-Dokumentation, wodurch Entwickler und Endnutzer die App-Funktionen direkt testen und verstehen können. Die Swagger UI wurde für eine intuitive Nutzung angepasst, um den Bedürfnissen aller Benutzergruppen gerecht zu werden. im Anhang B.1 [Abbildung 7: Benutzeroberfläche\(GUI\)](#) steht eine Übersicht zur Verfügung. Außerdem gibt es noch eine weitere Übersicht im Anhang B.1 [Abbildung 8: CRUD-Funktionalität](#) zuverfügung.

4.4 Pflichtenheft

Zum Abschluss der Entwurfsphase wurde ein Pflichtenheft erstellt. Dieses Dokument, verfasst aus der Sicht des Auftragnehmers, dokumentiert die technische Umsetzung der im Anhang [A.3: Lastenheft\(Auszug\)](#) beschriebenen Anforderungen. Ausgewählte Abschnitte des Pflichtenhefts findet sich im Anhang [A.4: Pflichtenheft\(Auszug\)](#). Hiermit kann am Ende des Projektes überprüft werden, ob alle Anforderungen an die Anwendung abgedeckt und ob diese auch wie vereinbart umgesetzt wurden sind.

5 Implementierungsphase

5.1 Erstellung des Projektes

Die Entwicklung des Projekts wurde unter Verwendung der Programmiersprache Python durchgeführt, wobei PyCharm als integrierte Entwicklungsumgebung (IDE) diente. PyCharm wurde aufgrund seiner umfangreichen Funktionalitäten für Python-Entwickler ausgewählt, einschließlich Codevervollständigung, effizienter Fehlerbehandlung und Unterstützung für die Versionskontrolle.

Entwicklungsprozess: Am Anfang der Implementierungsphase wurde das Projekt in PyCharm eingerichtet mit allen benötigten Bibliotheken. Für die API-Kommunikation mit dem Odoo-ERP-System kam speziell die XML-RPC-Bibliothek zum Einsatz. In der Hauptklasse des Projekts, benannt MyApplication, wurden alle notwendigen Einstellungen und Variablen mittels objektorientierter Programmierung gesetzt. Insbesondere wurden unter der Methode `def __init__(self, config_class=DevelopmentConfiguration)` alle wichtigen Variablen, sowohl Klassen- als auch Instanzvariablen, mit dem Schlüsselwort (`self`) verknüpft oder definiert, Abschnitt B.5 [Hauptklassen Strukturierung übersicht](#). In Python werden Variablen und Methodennamen standardmäßig in Kleinbuchstaben (small case) geschrieben, wobei Methoden mit dem Schlüsselwort `def` eingeleitet werden. Klassen werden in CamelCase (mit Großbuchstaben am Anfang jedes Wortes) benannt. Das Schlüsselwort `self` wird zum Initialisieren von Instanzattributen und zum Zugriff auf diese Attribute und Methoden innerhalb von Klassenmethoden verwendet. `self` bezieht sich immer auf das aktuelle Instanzobjekt. Der Name `self` ist dabei konventionell, kann aber prinzipiell frei gewählt werden. Standardmäßig wird (`self`) und (`cls`) verwendet.

Die `__init__` ist eine spezielle Methode in Python, die als Konstruktor fungiert. Sie wird aufgerufen, wenn ein neues Objekt der Klasse erstellt wird und um das Objekt zu initialisieren (Zuweisen der Startwerte). Der `self`-Parameter in `__init__` ist die erste Referenz auf die neu erstellte Instanz und wird verwendet, um Instanzvariablen zu initialisieren. Der Entwicklungsprozess gliederte sich in mehrere Schlüsselphasen.

- **Konzeptionierung:** Definition der grundlegenden Struktur der API und der Kommunikationsmechanismen mit dem Odoo-System. In dieser Phase wurden auch die Sicherheitsaspekte berücksichtigt, um die Authentifizierung und den sicheren Datenaustausch zu gewährleisten.
- **Implementierung der API-Endpunkte:** Entwicklung der spezifischen Funktionen der API, um die erforderlichen Datenoperationen (wie das Abrufen und Aktualisieren von Daten) zu unterstützen. Jeder Endpunkt wurde sorgfältig getestet, um die korrekte Interaktion mit dem Odoo-ERP sicherzustellen.
- **Nutzerverwaltung:** Wurde mittels einer SQLite-Datenbank realisiert, einschließlich Nutzerregistrierung, Login und rollenbasierter Zugriffskontrolle.
- **Einrichtung der Datenbankverbindung:** Erfolgte durch die Integration von Flask-SQLAlchemy, wozu die Installation der Bibliothek und die Konfiguration der Anwendung gehörten, um den Pfad zur SQLite-Datenbank anzugeben.
- **Integrationstests:** Dienten der Verifizierung der nahtlosen Integration aller Komponenten, einschließlich der Kommunikation mit dem Odoo-System und der Nutzerverwaltungsdatenbank.
- **Fehlerbehebung und Optimierung:** Wurden auf Basis der Integrationstestergebnisse durchgeführt, um Fehler zu korrigieren und die Anwendungsleistung zu verbessern.

5.2 Initialisierung und Start des Projekts (run.py):

Die Initialisierung des Projekts erfolgt über die run.py Datei, die als Einstiegspunkt dient. Sie konfiguriert und startet die Flask-Anwendung und öffnet automatisch einen Webbrowser, der auf die Homepage der Anwendung (<http://127.0.0.1:5000/home>) verweist. Durch den Einsatz eines Timers wird der Browser eine Sekunde nach dem Start der Anwendung geöffnet. Diese Datei prüft die Verfügbarkeit des Chrome-Browsers und fällt, falls nicht verfügbar, auf Microsoft Edge zurück, [Abbildung.9: Benutzer Freundlichkeit des Startpunkt](#) auf der seite (VIII).

Automatische Browseröffnung: Eine benutzerfreundliche Funktion, die den direkten Zugang zur Anwendungs-Startseite nach dem Start der Anwendung ermöglicht.

5.3 Hauptklasse und Systemkonfiguration (__init__.py):

MyApplicaiton Die Kernlogik und Konfiguration der Anwendung sind in der Klasse MyApplication definiert. Sie kümmert sich um die umfassende Einrichtung und Konfiguration der Flask-Anwendung [Abbildung.11: Hauptstruktur des Quell Code](#). Dies beinhaltet:

Flask-Anwendung und Konfiguration: Initialisierung der Flask-Anwendung mit spezifischen Einstellungen für die Entwicklungsumgebung.

Datenbankanbindung: Einrichtung der SQLite-Datenbank mittels SQLAlchemy für die Nutzerverwaltung und die Migrationsverwaltung mit Flask-Migrate [Abbildung.13: Datenbank Migration und API Einrichtung](#). Die Anwendung unterstützt eine SQLite-Datenbank, deren Schema und Session-Management durch Flask und SQLAlchemy gehandhabt werden.

API Einrichtung- und Route-Setup: Erstellung der Flask-RestPlus API mit spezifischen Namespaces für verschiedene Bereiche wie Odoo-Datenabfrage, Nutzerverwaltung und Hilfebereich. Definition von Endpunkten und Routen für die Nutzerinteraktion. Der Entwickler verwendet Flask-Restx, für die Sichreheitmaßnahmen einschließlich Versionierung, Titel, und Beschreibung. Sicherheitsmechanismen wie API-Keys und BasicAuth darunter hatte Der Entwickler besaß, während der Entwicklungszeit, Sicherheitsmechanismen, wie API-Keys und BasicAuth, um den Zugriff der Anwendung zu kontrollieren und um sicherzustellen, dass keine unbefugte Person Daten manipulieren kann.

Authentifizierungsmanagement: Durch HTTPBasicAuth und HTTPTokenAuth, um sowohl Basic Authentication, als auch Token-basierte Authentifizierung, zu ermöglichen.

Odoo-Kommunikation: Einrichtung der Verbindung und des Datenabgriffs von Odoo mittels spezifischer Funktionen und Modelle für die Interaktion mit dem ERP-System.

Die Struktur und Organisation des Codes verdeutlichen einen modularen Ansatz, der die Wartbarkeit und Erweiterbarkeit der Anwendung erleichtert. Die sorgfältige Aufteilung in Komponenten und die klare Trennung der Verantwortlichkeiten innerhalb des Codes sind beispielhaft für moderne Softwareentwicklungspraktiken.

5.4 Implementierung der Geschäftslogik

Die Geschäftslogik des Projektes umfasst die zentralen Funktionen und Prozesse, die notwendig sind, um die Anforderungen der Unternehmensressourcenplanung effizient zu erfüllen. Sie konzentriert sich auf die Integratinon und Manipulation von Daten aus dem Odoo-ERP-System, sowie die Verwaltung von Nutzerinteraktionen und -Authentifizierung. Die Implementierung erfolgte hauptsächlich in Python, unter Nutzung des Flask-Frameworks und weiteren unterstützenden Bibliotheken.

Kernfunktionalitäten: Das System integriert Odoo für Datenmanagement und Nutzerverwaltung, nutzt die XML-RPC-Schnittstelle für sichere Datenabfragen und ermöglicht Updates, sowie das Hinzufügen und Löschen

von Daten mit strengen Sicherheitsmaßnahmen. Mittels Middleware Secure API und SQLite wird der rollenbasierte Zugriff gesteuert, während fortschrittliche Authentifizierungsverfahren und Zugriffskontrollen die Sicherheit und Nutzerdifferenzierung verstärken. Die Automatisierung kritischer Funktionen wie der Anwesenheitserfassung trägt zur Effizienzsteigerung bei, sichert die Datenintegrität und unterstützt den Datenschutz.

5.5 Durchführung Einheit(Unittest), Integeration(Pytest) und Systemtest

Einheitentests und Integrationstests sind beides Verfahren im Softwaretestprozess, die unterschiedliche Aspekte der Anwendung überprüfen, auszuge nach dem Erfolgreiche EinheitenTest [Abbildung.15: Tests Durchführung](#), Darunter liegen folgende Tests, die Während des Projekt durchgeführt wurden:

Einheiten Test(Unittest): Wurden für jedes wichtige Modul erstellt, um deren Funktionalität und Integrität zu überprüfen, wobei alle Module erfolgreich bestanden haben.

Integeration(Pytest): Mit Pytest fokussierte sich der Entwickler auf die Überprüfung der korrekten Anbindung und das Zusammenspiel mit SQLite und der Integration mit dem Odoo-ERP-System. Dies stellt eine effiziente und fehlerfreie Datenübertragung zwischen den Systemen sicher.

Systemtests: Auf lokalen und virtuellen Umgebungen gelingt die Funktionalität, Stabilität und Systemzuverlässigkeit unter verschiedenen Bedingungen, einschließlich Extremfällen und alltäglicher Nutzung. Dies unterstreicht die Einsatzbereitschaft der Anwendung.

5.6 Fehlerbehandlung und Protokollierugn

Die Fehlerbehandlung und Protokollierung sind essentielle Bestandteile bei der Entwicklung robuster und wartbarer Software, um Probleme effizient zu identifizieren, analysieren und zu beheben. Dabei verwendete der Entwickler Tracebacks zur Debugging-Unterstützung und Logger für definierte Fehlerausgaben.

5.6.1 Fehlerbehandlung

Try-Except Blöcke: Entwickler hat Try-Except Blöcke verwendet, um potezielle Fehlerquellen gezielt abzufangen und entsprechende Maßnahmen zu ergreifen. Durch die Anwendung dieser Technologie kann das Programm stabil weiterlaufen und kontrollierte Fehlermeldungen ausgeben. Das hilft auch um den Fehlerkontext zu verstehen und die Ursache schnell zu identifizieren.

Ausnahme Kategorisierung: Für die Ausnahmen-Fehler-Fälle wurden spezifische Ausnahmenklassen verwendet. Dies hilft dabei, Fehlerquellen genauer zu identifizieren und spezifisch darauf zu reagieren.

Ressourcenmanagement: Verwendet Kontextmanager (with-Statement in Python) für das Management von Ressourcen (z.B. Dateioperationen und Netzwerkverbindungen), um sicherzustellen, dass Ressourcen ordnungsgemäß freigegeben werden, auch wenn Fehler auftreten.

5.6.2 Protokollierung

Konfiguration des Logging-Levels: Durch die Verwendung unterschiedlicher Logging-Levels (DEBUG, INFO, WARNING, ERROR, CRITICAL) konnte der Entwickler Ereignisse in unterschiedlichen Detailgraden protokollieren, [Abbildung.14 Fehler Protokollierung und Code-Dokumentation](#). Dies ermöglicht eine effektive Fehleranalyse und hilft dabei, die Logdateien übersichtlich zu halten.

5.7 Abnahme und Qualitätssicherung

Das Projekt unterlag einer sorgfältigen Abnahme durch die Fachabteilung IAD, welche es auf Übereinstimmung mit den gestellten Erwartungen hin überprüfte. Nach dieser umfassenden Beurteilung wurde das Projekt erfolgreich abgenommen.

6 Documentation

Im Rahmen dieses Projekts wurde eine umfangreiche Dokumentation durch Latex Framwork erstellt, die sich in drei Hauptbereiche gliedert: Nutzerdokumentation, allgemeine Projektdokumentation und Quellcodedokumentation. Aufgrund der Beschränkung des Umfangs auf weniger als 15 Seiten wollte der Entwickler nicht zu tief in die Details gehen, obwohl viel mehr zu erzählen gewesen wäre. Jeder dieser Bereiche zielt darauf ab, unterschiedliche Aspekte des Projekts zu beleuchten und sowohl Benutzern als auch Entwicklern einen detaillierten Einblick zu gewähren.

Die Projekt-Dokumentation bietet eine Nutzeranleitung in PDF für einfache Handhabung, eine Übersicht der Nutzeranleitung im Anhang.A [Abbildung.6: Nutzeranleitung](#), eine Übersichtsdokumentation zu Zielen und Aufbau des Projekts, sowie eine detaillierte Quellcodedokumentation. Diese Elemente erleichtern das Verständnis, helfen neuen Entwicklern beim Einstieg und sichern die Wartbarkeit des Codes.

Diese strukturierte Dokumentation gewährleistet, dass alle Projektbeteiligten und Nutzer die benötigten Informationen effizient und zielgerichtet finden können.

7 Fazit

Die Implementierung der Flask-RESTX API als Middleware zwischen externen Systemen und Odoo hat sich als eine effektive Lösung zur Verbesserung der Sicherheit und Effizienz im Datenaustausch erwiesen. Durch die sorgfältige Planung und Implementierung der Sicherheitsmechanismen wurde sichergestellt, dass die Datenkommunikation den höchsten Sicherheitsstandards entspricht.

7.1 Soll-/IST-Vergleich

Der Vergleich zeigt, dass die meisten Projektziele erreicht und teilweise sogar übertroffen wurden. Die Leistungsindikatoren bestätigen eine effektive Zielerreichung, mit Abweichungen, die wertvolle Einblicke für zukünftige Optimierungen bieten. Trotz einiger unvorhergesehener Herausforderungen spiegeln die Ergebnisse eine starke Übereinstimmung mit den ursprünglichen Erwartungen wider, unterstreichen den Erfolg des Projekts und legen den Grundstein für kontinuierliche Verbesserungen. Siehe Tabelle .[Tabelle 5: Soll-/IST-Vergeleichung](#) für den Soll-/IST-Vergleich.

Projektphase	Soll (Stunden)	IST (Stunden)	Differenz
I. Analysephasen	8h	10h	+2h
II. Planungsphase	10h	10h	0h
III. Realisierung	32h	25h	-7h
IV. Testphase	12h	15h	+3h
V. Dokumentation	18h	20h	+2h
Gesamt	80 Stunden	80 Stunden	0 Stunden

Tabelle 6: Soll-/Ist-Vergleich der Projektphasen

Literaturverzeichnis

flaskDocumentation:

title = "Flask Documentation", author = "Pallets Projects", year = 2023, note = Accessed: 2023-02-22
url = "<https://flask.palletsprojects.com/>"

sqlite:

title = SQLite,url = <https://sqlite.org>, note = Accessed: 2024-02-21

odoo:

title = Odoo, url = <https://www.odoo.com>, note = Accessed: 2024-02-23

python310

title = Python310, version = 3.10, note = Accessed: 2024-02-21
url = <https://www.python.org/downloads/release/python-3100/>,

python Docs:

title = Documentations, note = Accessed: 2024-02-21
url = <https://docs.python.org/3/library/sqlite3.html#reference>,

GitHub:

title = Projekt Dokumentation Latex-Versions Verwalten, note = Accessed: 2024-02-21,
url = https://github.com/Feraidon-Mohammadi/Feri_Projects/,

pycharm:

title = PyCharm, subtitle = Community and Professional Editions,
url = <https://www.jetbrains.com/pycharm/>, note = Accessed: 2024-02-21

chrome:

title = Google Chrome, url = <https://www.google.com/chrome/>, note = Accessed: 2024-02-19

Udemy Software Test:

title = Automated Software Testing with python, year = 2024, note = Accessed: 2024-02-27
url =<https://www.udemy.com/course/automated-software-testing-with-python/>

excel:

title = Microsoft Excel, note = Accessed: 2024-02-29
url = <https://www.microsoft.com/en-us/microsoft-365/excel>,

office-pro:

title = Microsoft Office Professional, note = Accessed: 2024-02-19,
url = <https://www.microsoft.com/en-us/microsoft-365/get-started-with-office-2019>,

msteams:

title = Microsoft Teams, note = Accessed: 2024-02-19,
url = <https://www.microsoft.com/en/microsoft-teams/group-chat-software>,

Flask-Odoo Guids:

title = Flask-Odoo,url = <https://pypi.org/project/Flask-Odoo/>, note = Accessed: 2024-02-21,

DigitalOcean:

title = DigitalOcean, note = Accessed: 2024-02-21,
url = <https://www.digitalocean.com/community/tags/python>,

Latex(MiKTeX):

title = Latex(MiKTeX),url = <https://miktex.org/download>, note = Accessed: 2024-02-20,

A Anhangsverzeichnis

A.1 Detaillierte Zeitplanung

Phase / Aufgabe	Zeit (h)
Analysephase	8
1. Durchführung der Ist-Analyse	2
1. Durchführung der Soll-Analyse	2
2. Durchführung der Wirtschaftlichkeitsanalyse	2
3. Erstellung des Lastenheftes	2
Planungsphase	10
1. Zielplattform festlegen / Architekturentwurf	3
2. Funktionsplanung	2
3. Kosten/Nutzenanalyse	3
5. Pflichtenheft erstellen	2
Realisierung-/Implementierungsphase	32
1. Entwicklungsumgebung einrichten	2
2. Anlegen der Datenbank und Datenbankmigration	2
3. Implementierung der Datenbankstruktur, Benutzer Rechte und Rollen	5
4. Implementierung der Geschäftslogik	23
4.1. Implementierung der API mit Flask-Restx	3
4.2. Implementierung der Benutzer Authorisierung und Authentifizierung Logik	3
4.3. Implementierung der Odoo Integration	3
4.4. Implementierung der Odoo Routes-/Endpunkten-/Ressourcen	5
4.5. Implementierung der User Routes-/Endpunkten-/Ressourcen	3
4.7. Implementierung der Response Header	2
4.8. Implementierung der Mail und Support Logikal Services	2
4.9. Implementierung der API Dokumentations-Schema	2
Testphase	12
1. Durchführung Unittests	5
2. Durchführung Integrations-Tests	2
3. Durchführung Systemtests	2
4. Fehlerbehebung und Identifizierung	2
5. Abnahme durch die Fachabteilung	1
Erstellen der Dokumentationen	18
1. Erstellen der Projektdokumentation	10
2. Erstellen der Entwicklerdokumentation	3
3. Erstellen der Benutzerdokumentation	5
Gesamt	80

Tabelle 8: Detaillierte Zeitplanung

A.2 Verwendete Ressourcen

Ein dedizierter Arbeitsbereich, der die konzentrierte und professionelle Entwicklung des Projekts ermöglicht mit folgender Hard-und Software:

Hardware:

- **Desktop mit zwei Bildschirmen:** Die Doppelbildschirm-Konfiguration erleichtert die gleichzeitige Betrachtung von Code und Dokumentation.
- **Maus, Tastatur, Headset:** Eingabegeräte und Kommunikationswerkzeuge, die für die tägliche Entwicklungsarbeit und Absprachen im Team essentiell sind.

Software:

- **Windows 10 Enterprise:** Das Betriebssystem, auf dem alle Entwicklungswerkzeuge laufen.
- **PyCharm Community Edition 2023:** Eine integrierte Entwicklungsumgebung (IDE) für Python, die umfangreiche Funktionen zur Codeerstellung, -Verwaltung der debugging bietet.
- **GitHub:** Wird als Versionsverwaltungssystem eingesetzt, um Änderungen an der LaTeX-Dokumentation des Projekts nachzuverfolgen und zu verwalten.
- **LaTeX (MiKTeX):** Textsatzsystem zur Erstellung professioneller Dokumentationen des Projekts.
- **Microsoft Office Professional 2023:** Bürosoftware für die Erstellung von Notizen und die Verwaltung projektbezogener Dokumente.
- **Python 3.10:** Die Programmiersprache, in der das Projekt entwickelt wurde. Python 3.10 bietet die aktuellsten Features und ist für die Implementierung des Projekts erforderlich.
- **Microsoft Teams:**
Wird für die Kommunikation zwischen dem Entwickler, dem Anwendungsentwickler und dem Geschäftsführer des IAD eingesetzt. Diese Plattform ermöglicht regelmäßige Abstimmungen, die Überprüfung des Projektfortschritts und trägt zur Qualitätssicherung bei, indem sie sicherstellt, dass das Projekt den Anforderungen entspricht.

Personal:

- **Entwickler:** Verantwortlich für die vollständige Umsetzung des Projekts, einschließlich Codierung, Tests und Debugging.
- **Anwendungsentwickler:** Führt die Abnahme und das Review des Projekts durch, um sicherzustellen, dass es den Anforderungen entspricht.
- **IAD-Leiter:** Auftraggeber, Abstimmung der Oberfläche und Abnahmedes Projektes.

A.3 Lastenheft

Die Anwendung muss folgende anforderungen Erfüllen:

Unter Anwendung der MoSCoW-Priorisierungsmethode wurden die Anforderungen im Lastenheft kategorisiert und entsprechend ihrer Wichtigkeit geordnet. Hier ist, wie der Abschnitt Anforderungen unter Berücksichtigung der Must, Should, Could, Would (MoSCoW)-Methode strukturiert werden könnte:

Must-have (Muss-Kriterien):

- **Integration mit Odoo:** Die Anwendung muss eine nahtlose Integration mit dem Odoo-ERP-System bieten, um den Datenzugriff und die Datenmanipulation zu ermöglichen.
- **Authentifizierung mit JWT:** Jeder Endpunkt der Anwendung muss die Authentifizierung mittels JWT unterstützen, um die Sicherheit zu gewährleisten.
- **Benutzerverwaltung:** Es müssen Endpunkte existieren, die es einem Superuser ermöglichen, Benutzerkonten hinzuzufügen, zu aktualisieren und zu löschen.
- **Datenbanknutzung:** Die Anwendung muss SQLite zur Verwaltung spezifischer Benutzerrechte einsetzen.
- **Swagger Integration:** Die API muss eine Integration von Swagger mittels Flask-RestX aufweisen, um eine interaktive Dokumentation zu ermöglichen. Dies ermöglicht es den Nutzern, die Endpunkte direkt über die Benutzeroberfläche zu testen und erleichtert das Verständnis der API-Funktionalitäten.

Should-have (Soll-Kriterien):

- **Objektorientierte Programmierung:** Das Projekt sollte OOP-Prinzipien anwenden, um modulare und wartbare Code-Basen zu gewährleisten.
- **Dokumentation der Endpunkte:** Eine klare und verständliche Dokumentation für jeden Endpunkt sollte bereitgestellt werden, um die Nutzung der API zu erleichtern.

Could-have (Kann-Kriterien):

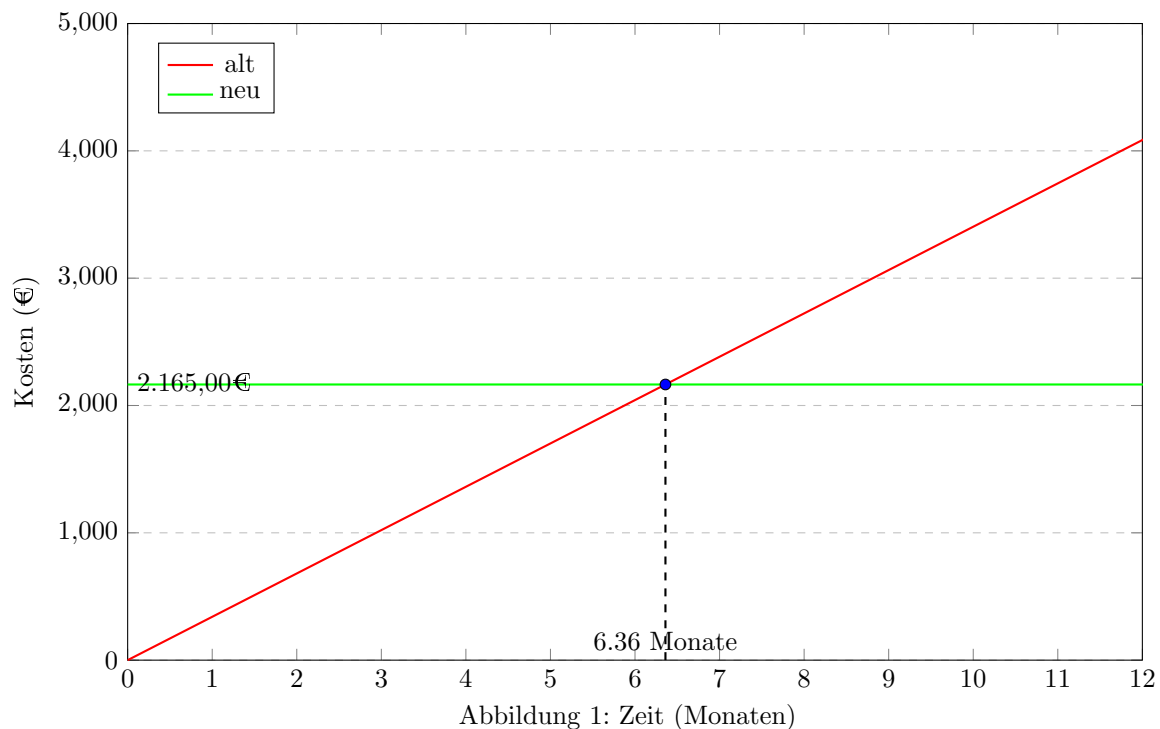
- **Erweiterte Sicherheitsfeatures:** Zusätzliche Sicherheitsmaßnahmen wie Rate Limiting und Zugriffskontrollen könnten implementiert werden, um die API weiter abzusichern.
- **Benutzerfreundliche Oberfläche:** Eine grafische Oberfläche für die Verwaltung der Benutzer könnte entwickelt werden, um die Benutzererfahrung zu verbessern.

Would-have (Würde-Kriterien):

- **E-Mail-Verifizierungsfunktion:** Optional könnte eine Funktion eingeführt werden, die bei der Registrierung eine Verifizierungs-E-Mail verschickt. Die Speicherung der Benutzerdaten in SQLite erfolgt nur nach erfolgreicher Bestätigung der E-Mail.
- **Konditionale Datenpersistenz:** Nicht verifizierte Benutzerkonten würden vorerst nicht in der Datenbank gespeichert, um die Datenintegrität zu gewährleisten. Diese Funktion bleibt für zukünftige Updates vorbehalten.

A.4 Diagram-Amortisationsdauer

Amortisationsdauer: Der Schnittpunkt der beiden Geraden gibt den Zeitpunkt der Amortisation an.



A.5 Vorgehensweise

V-Model:

Der Entwickler hat das folgende Modell als Vorgehensweise für den Ablauf seines Projekts ausgewählt.

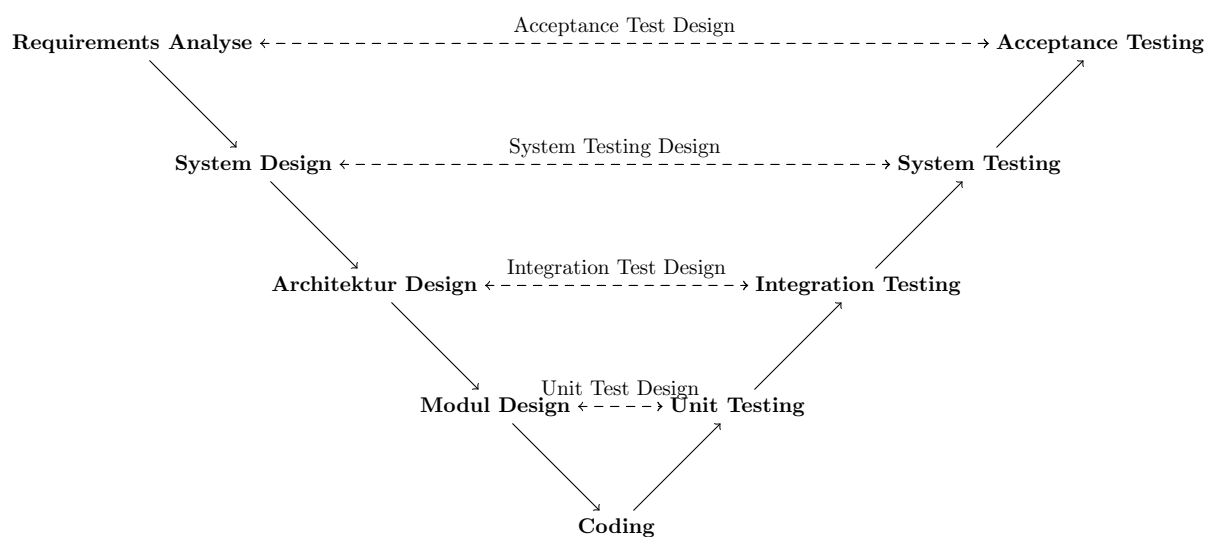


Abbildung 2: Vorgehensmodell

A.6 Aktivitäts-Diagramm

Grafische Darstellung der Benutzeraktivitäten:

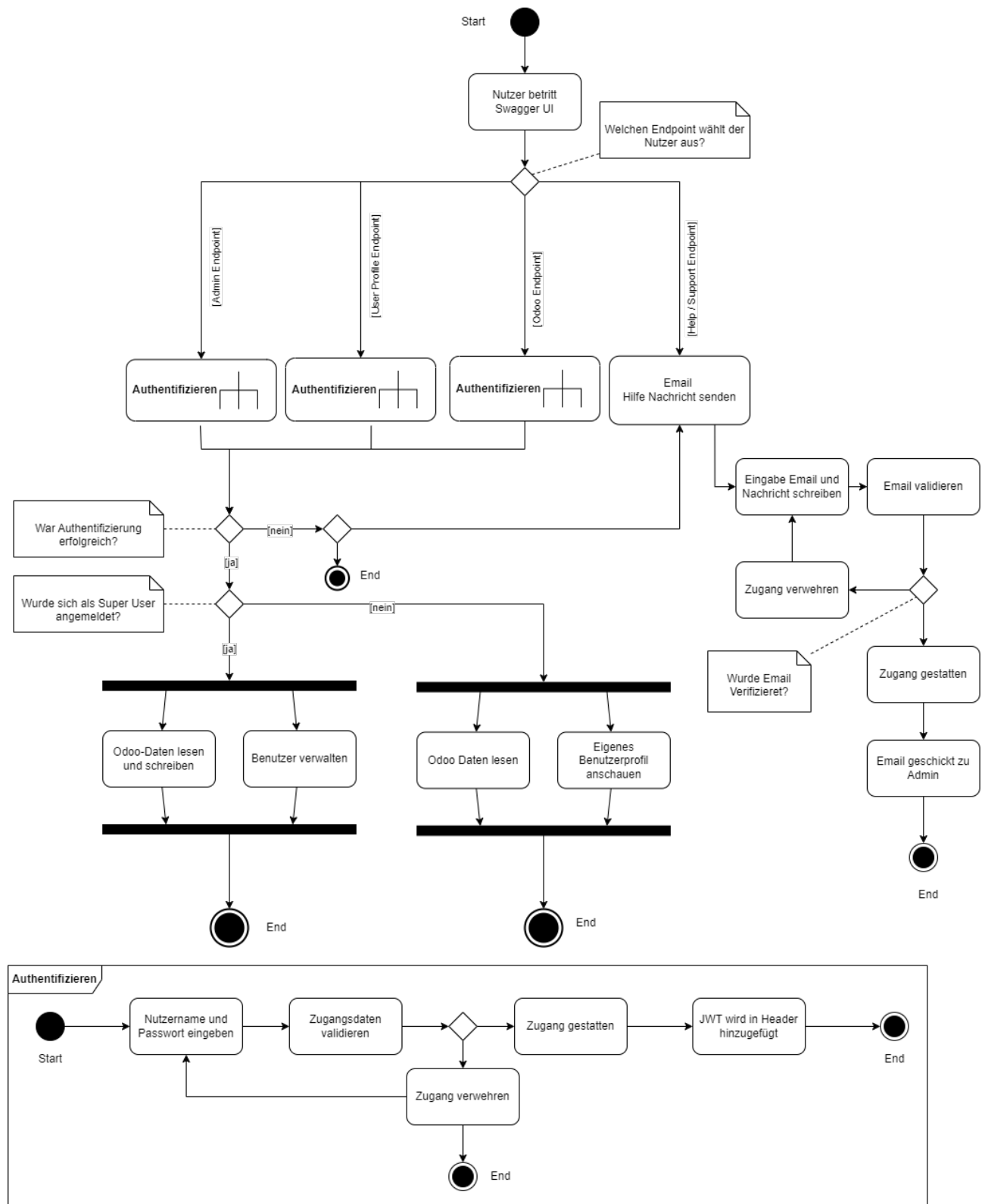


Abbildung 3: Aktivitäts-Diagramm

A.7 Klassen-Diagramm

Da es in Python keine privaten oder öffentlichen Klassen gibt, habe ich diese nicht gekennzeichnet.

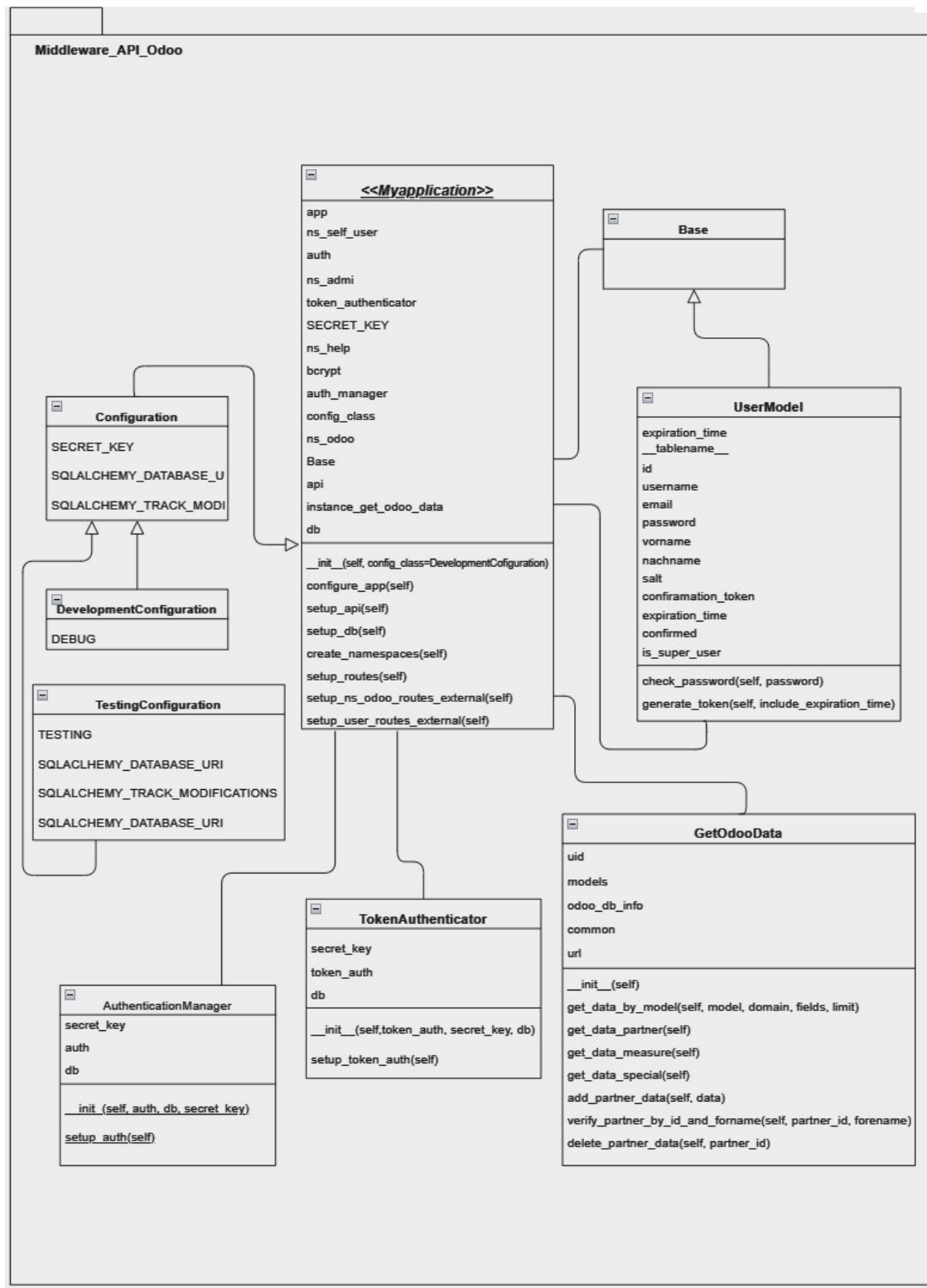


Abbildung 4: Klassendiagramm

A.8 Pflichtenheft (Auszug)

1-Zielbestimmung

- **Projektziel:** Implementierung einer Python/Flask-RestX-Anwendung mit Odoo-Integration zur Datenverwaltung und Nutzerregistrierung.

2-Funktionale Anforderungen

- **Swagger-Dokumentation:** Die Erstellung interaktiver API-Dokumentation unter Verwendung von Flask-RestX Swagger, um Endpunkte zu dokumentieren und testbar zu machen.

3-Nicht-funktionale Anforderungen

- **Benutzerfreundlichkeit:** Gewährleistung einer intuitiven Nutzerführung innerhalb der Swagger-Dokumentation.
- **Performance:** Sicherstellung einer angemessenen Antwortzeit der API.

4-Systemarchitektur

- **Modulare Struktur:** Aufbau der Anwendung in separaten Modulen für eine klare Trennung der Verantwortlichkeiten.

5-Entwurf und Implementierung

- **Datenbankdesign:** Einsatz von SQLite zur Verwaltung der Benutzerrechte.
- **Sicherheitskonzept:** Anwendung von JWT für die Authentifizierung an den API-Endpunkten.

6-Benutzerschnittstelle

- **API-Endpunkte:** Bereitstellung von Endpunkten für die Kommunikation zwischen der Anwendung und Odoo.

7-Testkonzept

- **Unit Tests:** Entwicklung und Durchführung von Unit Tests für alle Komponenten.
- **Integrationstests:** Überprüfung der korrekten Interaktion der Module und der Integration mit Odoo.

8-Abnahme

- **Testfälle:** Definition von Testfällen, die bei der Abnahme durchgeführt werden müssen.
- **Abnahmekriterien:** Spezifikation der Kriterien, die für die Abnahme der Anwendung erfüllt sein müssen.

9-Inbetriebnahme

- **Deployment:** Beschreibung des Deployments und der notwendigen Schritte zur Inbetriebnahme der Anwendung.

10-Qualitätssicherung

- **Code-Reviews:** Regelmäßige Code-Reviews zur Sicherung der Codequalität.
- **Sicherheitstests:** Durchführung von Sicherheitstests zur Verifizierung der Authentifizierungsfunktionen.

A.9 Auswahl der Programmiersprache Python

Der Entwickler hat aus folgenden Grund die Python Programmiersprache ausgewählt:

Kriterium	Python	Java	C#	Ruby	JavaScript
1- Integration mit Odoo	5	3	3	4	3
2- Unterstützung für SQLite	5	4	4	4	4
3- Eignung für RESTful API	5	4	4	4	4
4- Rapid Prototyping	5	3	3	4	4
5- Verfügbare Bibliotheken	5	4	4	4	4
6- Lernkurve	5	3	3	4	4
7- Community und Support	5	4	4	4	5
8- Entwicklungs-Geschwindigkeit	5	3	3	4	4
9- Performance	3	4	4	3	3
10- Wartbarkeit	5	4	4	4	3
11- Cross-Plattform-Fähigkeit	5	4	3	4	5
Gesamt:	53	40	39	43	43

Tabelle 9: Vergleich zu anderen Programmiersprachen

- 1- Odoo ist in Python geschrieben, daher ist die Integration natürlicher und direkter.
- 2- Python hat eine ausgezeichnete eingebaute Unterstützung für SQLite, andere Sprachen benötigen zusätzliche Treiber.
- 3- Flask und Django sind beides sehr ausgereifte Frameworks speziell für das Erstellen von RESTful APIs in Python.
- 4- Python's einfache Syntax und Dynamik erleichtern schnelles Prototyping.
- 5- Python hat eine riesige Sammlung von Bibliotheken für fast jeden Zweck, insbesondere für Webentwicklung und Datenintegration.
- 6- Python ist bekannt für seine klare Syntax und gute Lesbarkeit, was die Einarbeitung erleichtert.
- 7- Python hat eine sehr aktive Community und ein breites Angebot an Dokumentationen und Foren.
- 8- Python ermöglicht eine schnelle Entwicklung durch eine klare Syntax und kraftvolle Frameworks.
- 9- Während Python nicht immer die schnellste Laufzeit hat, ist es in der Praxis selten ein limitierender Faktor und kann durch Tools wie PyPy verbessert werden.
- 10- Python's einfache Syntax und Strukturierung fördern eine gute Wartbarkeit.
- 11- Python läuft auf allen großen Betriebssystemen ohne Änderungen.

A.10 Ablaufzeitplan

Das Projekt wurde innerhalb folgenden Zeitplans durchgeführt:

Std.	Start_datum	End_datum	Aufgabe/Vorgan	19.02.2024	20.02.2024	21.02.2024	22.02.2024	23.02.2024	24.02.2024	25.02.2024	26.02.2024	27.02.2024	28.02.2024	29.02.2024	01.03.2024	02.03.2024	03.03.2024	04.03.2024
				Mo	Di	Mi	Do	Fr	Sa	So	Mo	Di	Mi	Do	Fr	Sa	So	Mo
8.00	19.02.2024	19.02.2024	Analysephase															
2.00	19.02.2024	19.02.2024	Ist-Analyse	2														
2.00	19.02.2024	19.02.2024	Soll-Analyse	2														
2.00	19.02.2024	19.02.2024	Wirtschaftlichkeitsanalyse	2														
2.00	19.02.2024	19.02.2024	Lastenheft	2														
10.00	20.02.2024	21.03.2024	Planungsphase															
3.00	20.02.2024	20.02.2024	Architekturentwurf		3													
2.00	20.02.2024	20.02.2024	Funktionsplanung		2													
3.00	20.02.2024	20.02.2024	Kosten/Nutzenanalyse		3													
2.00	21.02.2024	21.02.2024	Pflichtenheft erstellen			2												
32.00	21.02.2024	27.03.2024	Implementierungsphase															
2.00	21.02.2024	21.02.2024	Einrichtung Entwicklungsumgebung			2												
2.00	21.02.2024	21.02.2024	Anlegen der Datenbank			2												
5.00	21.02.2024	22.02.2024	Datenbankstruktur			2	3											
3.00	22.02.2024	22.02.2024	Implementierung API logik				3											
3.00	22.02.2024	23.02.2024	benutzer Rechtrollen				2	1										
3.00	23.02.2024	23.02.2024	Odoo Intergration					3										
5.00	23.02.2024	26.02.2024	Odoo Endpunkte					4		1								
3.00	26.02.2024	26.02.2024	Users Endpunkte							3								
2.00	26.02.2024	26.02.2024	Response Header Sicherheitmaßnahmen							2								
2.00	26.02.2024	26.02.2024	Mail und support logik							2								
2.00	27.02.2024	27.02.2024	API Dokumentaions Shema								2							
12.00	27.02.2024	02.03.2024	Testphase															
5.00	27.03.2024	27.03.2024	Unittest								5							
2.00	27.03.2024	28.03.2024	Integrations Test								1	1						
2.00	28.03.2024	28.03.2024	System Test									2						
2.00	28.03.2024	28.03.2024	Fehlerbehebung									2						
1.00	28.03.2024	28.03.2024	Qualitätsicherung und Abnahme									1						
18.00	28.02.2024	01.03.2024	Dokumentation															
5.00	28.02.2024	29.02.2024	Projektdokumentation									1	4					
5.00	29.03.2024	01.03.2024	Projektdokumentation										4	1				
3.00	01.03.2024	01.03.2024	Entwicklerdomentation											3				
5.00	01.03.2024	04.03.2024	benutzerdokumenation											4				1
80.00	19.02.2024	02.03.2024	Geplante Tagesstunden	8	8	8	8	8	0	0	8	8	8	8	8	0	0	1

Abbildung 5: Ablaufzeitplan des Projekts.

7.11 Nutzer-Anleitung

eine Strukturierte Nutzeranleitung steht zur Verfügung:

Hier ist eine Übersicht der ersten Seite, extrahiert aus der Benutzerdokumentation.



Abbildung 6: Nutzer-Anleitung

B Abbildungsverzeichnis

B.1 Benutzeroberfläche

Eine Übersicht aus der API-GUI während der Laufzeit des Browsers mit 4 Haupt-Panels, Userverwaltungs-Panel, Odoo-Datenverwaltungs-Panel, User-Profile-Panel und Support-Panel.

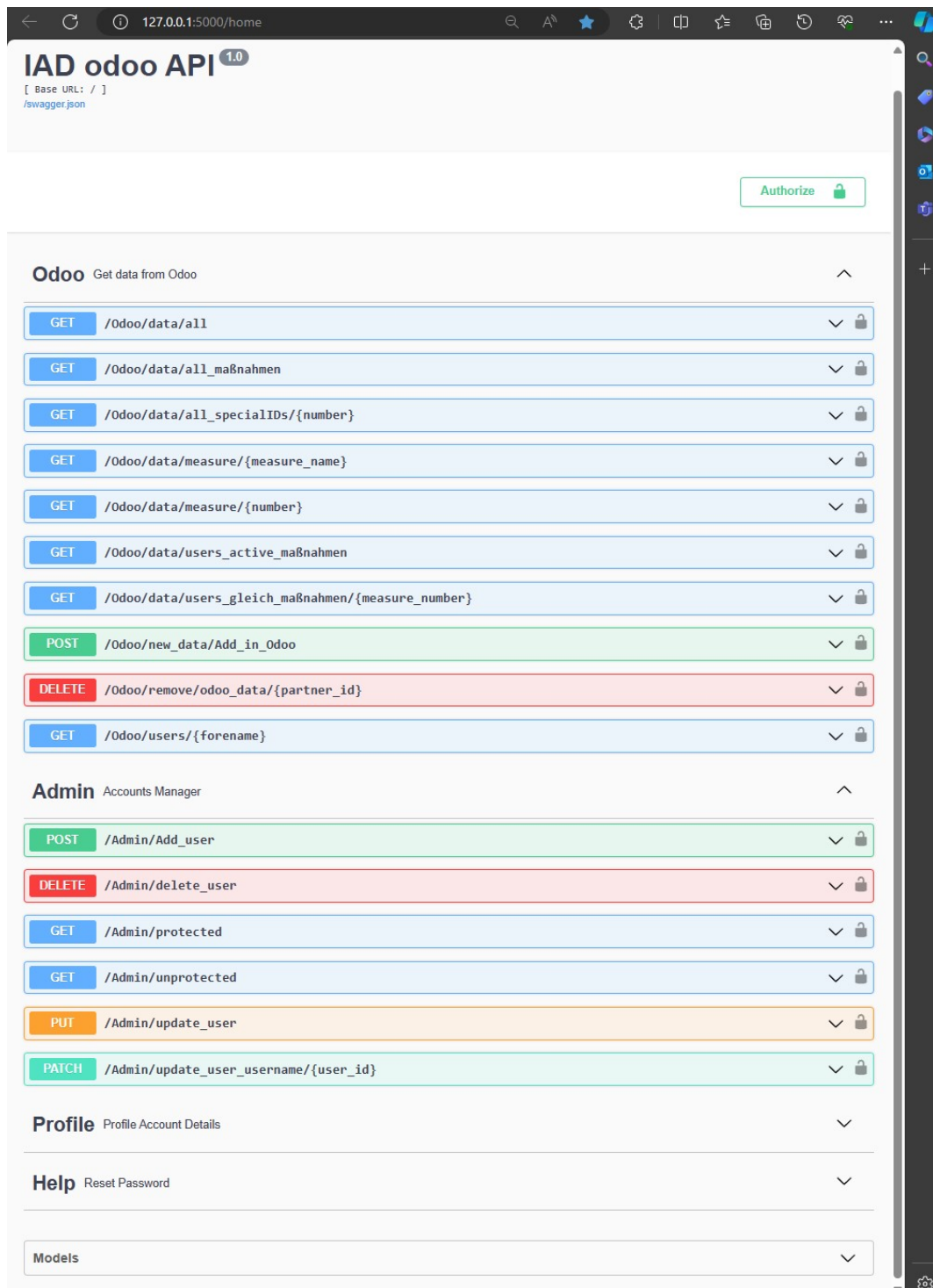


Abbildung 7: Benutzeroberfläche(GUI)

B.2 Admin-Haupt-Userverwaltung-Panel

Höchstmögliche Sicherheitsmechanismen wurde verwendet für alle Endpunkte, besonders bei diese Endpunkten.

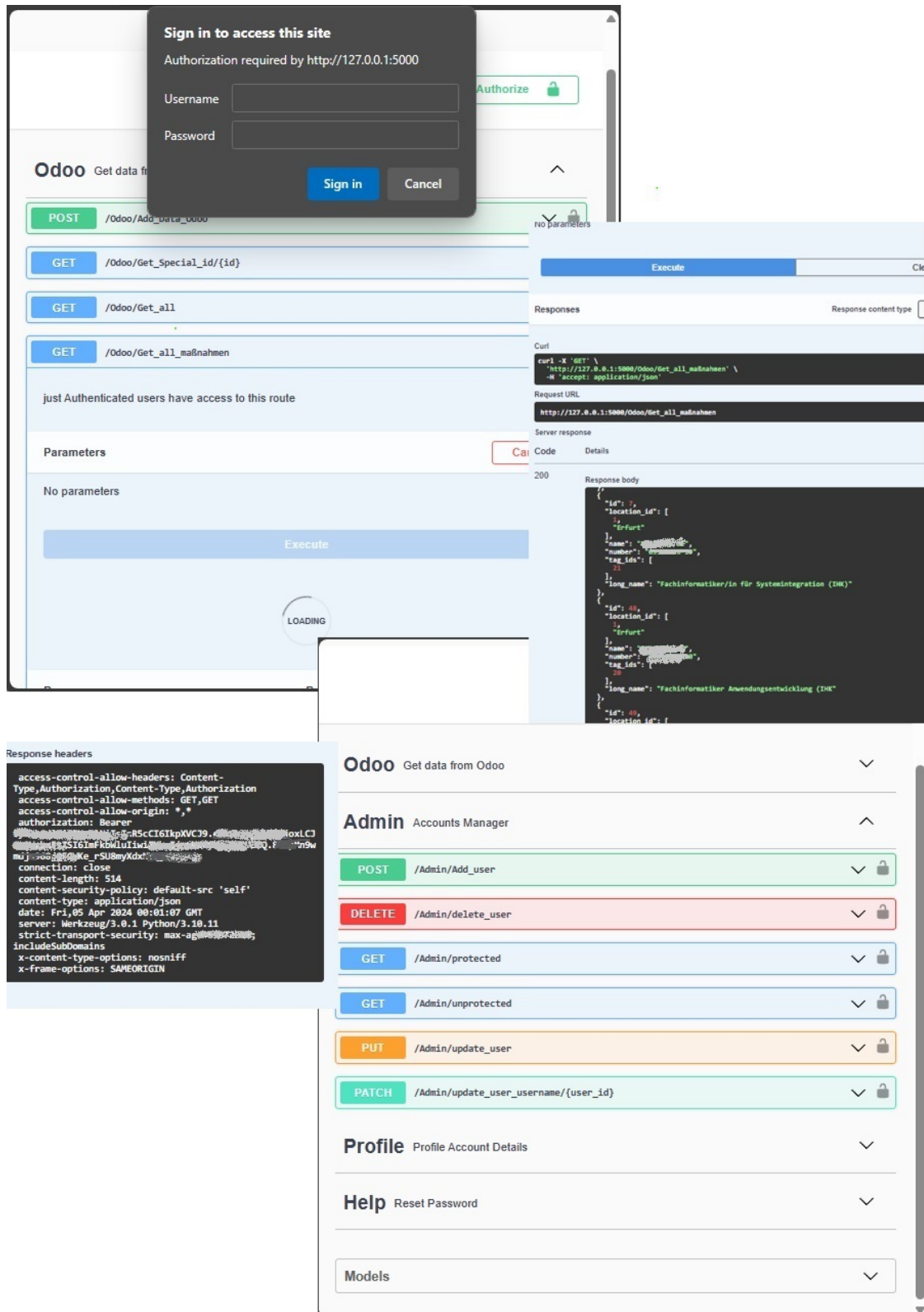


Abbildung 8: CRUD-Funktionalitäten (Create- Read-Update-Delete)

B.3 Projekt Startpunkt

Run.py mit einfachen Startpunkt um die Benutzerfreundlichkeit zu erhöhen.

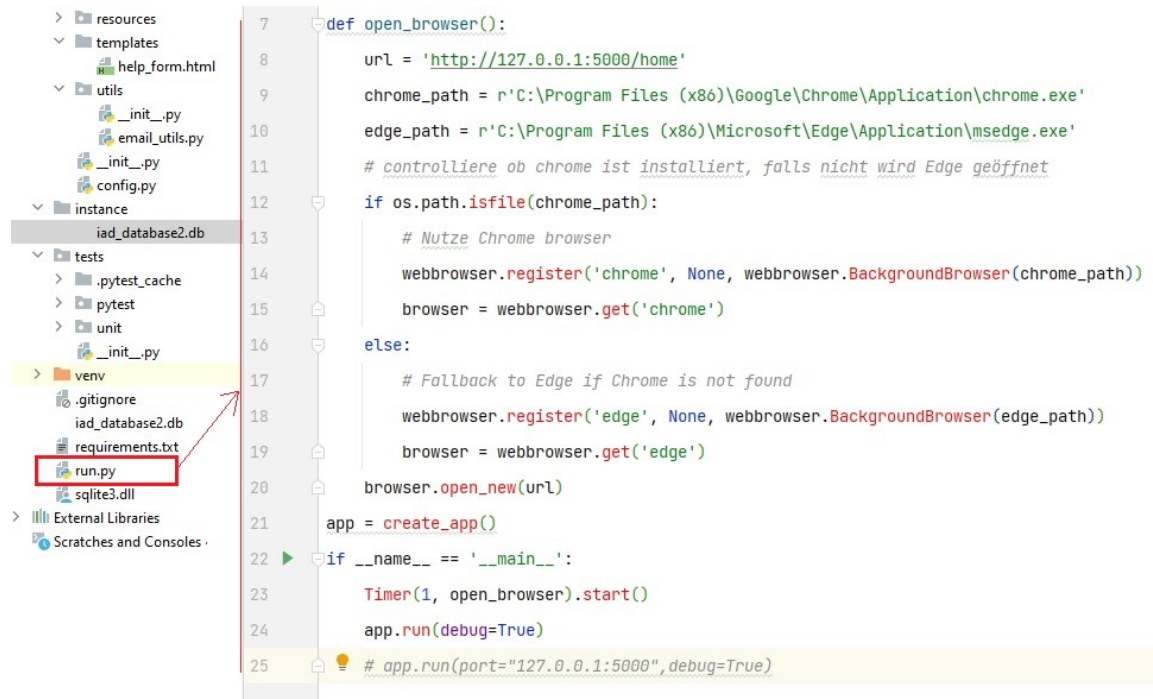


Abbildung 9: Startpunkt des Projekts mit benutzerfreundlicher Schnittstelle

B.4 Benutzer Anmeldung Prozess auszuge

Auszuge ausdem console, während einloggen und update durch beschränkte Benutzer noch Benutzer mit erweiterte Rechte, erst mit falsche password und dann mit richtige.

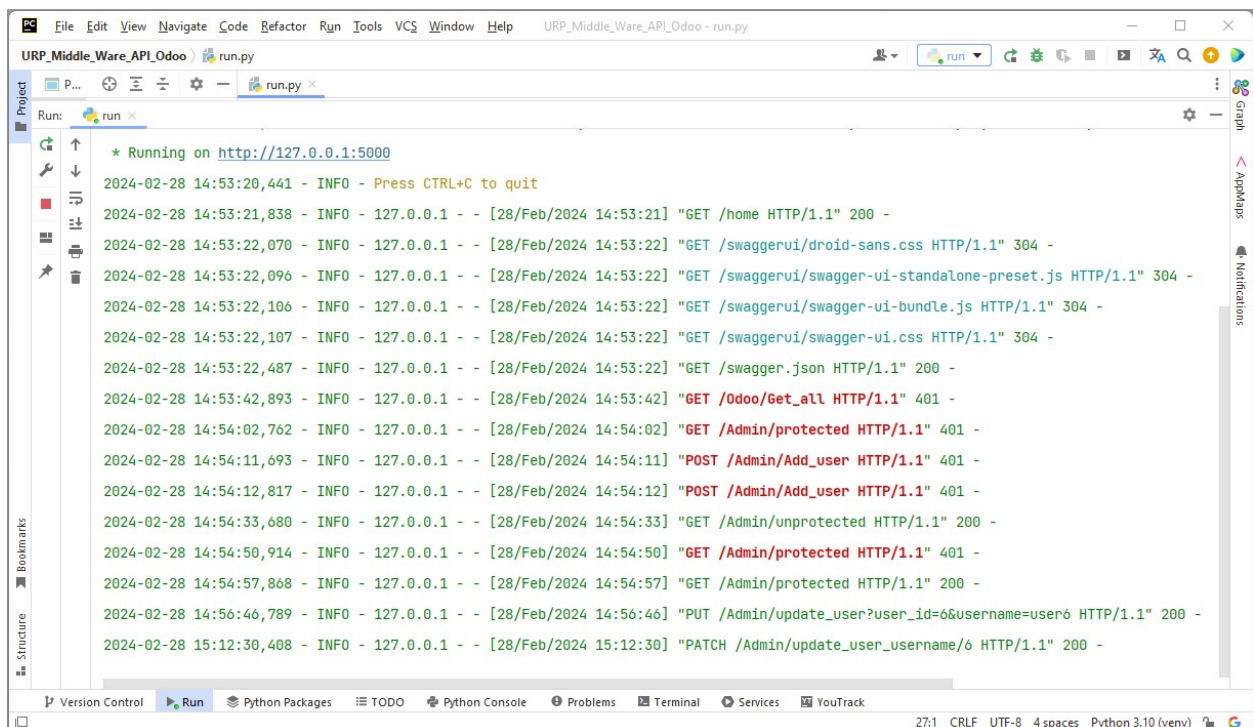


Abbildung 10: Projekt Auszuge-bevor und nach login

B.5 Haupt-Klassen-übersicht

```
1  class MyApplication:
2
3      def __init__(self, config_class=DevelopmentConfiguration):
4          self.app = Flask(__name__, template_folder="templates")
5          self.config_class = config_class
6
7          """ Odoo data for using unit test needed """
8          if GetOdooData == 200:
9              self.instance_get_odoo_data = GetOdooData()
10         else:
11             self.instance_get_odoo_data = None
12             """ when odoo mit authentication connecting, need this,
13             otherwise should use upper (if statement to by pass it)
14             """
15             # self.instance_get_odoo_data = GetOdooData()
16
17         self.setup_api()
18         MyApplication.api = self.api # Set the class variable
19         self.db = SQLAlchemy()
20         MyApplication.db = self.db
21         self.bcrypt = Bcrypt(self.app)
22         # self.mail = Mail(self.app) """ fuer die zukunft mail server hinzufügen
23         """
24         self.auth = HTTPBasicAuth()
25         self.token_auth = HTTPTokenAuth(scheme='Bearer')
26         # database und Migration
27         self.Base = declarative_base()
28         self.configure_app()
29         self.setup_db()
30         self.create_namespaces()
31         self.setup_routes() # aufruf setup_routes zu register route
32         self.setup_user_routes_external()
33         # instances of authentication and token authenticator
34         self.auth_manager = AuthenticationManager(self.auth, self.db, self.
35             SECRET_KEY)
36         self.auth_manager.setup_auth()
37         self.token_authenticator = TokenAuthenticator(self.token_auth, self.
38             SECRET_KEY, self.db)
39         self.token_authenticator.setup_token_auth()
40
41         """ Odoo data """
42         self.setup_ns_odoo_routes_external()
43
44     def configure_app(self):
45         self.SECRET_KEY = self.app.config['SECRET_KEY']
46         self.app.config.from_object(self.config_class)
47         self.app.secret_key = self.app.config.get('SECRET_KEY', self.SECRET_KEY)
48
49     def setup_api(self):
50         self.api = Api(self.app, version='1.0', title='IAD odoo API', doc="/home"
51             ,
52             authorizations={ 'apiKey': { 'type': 'apiKey', 'in': '
53                 header', 'name': 'Authorization', }, 'basicAuth': { '
54                 type': 'basic', }, },
55             security=['BearerAuth', 'basicAuth'])
```

Abbildung 11: Hauptstruktur des Quell Code

B.6 Datenbankstruktur

Implementierung der Database wurde in folgende Klass strukturiert zusätzlich login sicherheit konzept zu erstellung des JWT.

```
1 class UserModel(Base):
2     __tablename__ = 'super_user'
3
4     id = Column(Integer, primary_key=True, autoincrement=True)
5     username = Column(String, nullable=False, unique=True)
6     email = Column(String, nullable=True)
7     password = Column(String, nullable=False)
8     vorname = Column(String, nullable=False)
9     nachname = Column(String, nullable=False)
10    salt = Column(String, nullable=False)
11    confirmation_token = Column(String, unique=True, nullable=True)
12    expiration_time = Column(DateTime, nullable=True)
13    confirmed = Column(Boolean, nullable=True)
14    is_super_user = Column(Boolean, default=False)
15
16    def check_password(self, password):... \# aus sicherheit grunden duerfte ich
17        diesespart nicht veroeffentlichen
18
19    def generate_token(self, include_expiration=True):... \# aus sicherheit
20        grunden duerfte ich dieses nicht veroeffentlichen
21
22
23 if os.environ.get('FLASK_ENV') != 'testing':
24     \# Only create tables if not in testing environment
25     database_file = 'iad_database2.db'
26     engine = create_engine(f'sqlite:/// {database_file}', echo=False)
27     \# Create super_users when it not exists
28     Base.metadata.create_all(bind=engine)
29
30
31 def create_user_models(ns_admin, ns_self_user, ns_help):
32     users_model = ns_admin.model('SuperUser_schema', {
33         'id': fields.Integer(readOnly=True, description='The unique identifier'),
34         'username': fields.String(required=True, description='Username of the
35             super user'),
36         'email': fields.String(description='Email of the super user'),
37         'password': fields.String(required=True, description='Password of the
38             super user',
39             attribute='password_hash'),
40         'vorname': fields.String(required=True, description='First name of the
41             super user'),
42         'nachname': fields.String(required=True, description='Last name of the
43             super user'),
44         'is_super_user': fields.Boolean(default=False,
45             description='Indicates if the user is a
46             super user')
47     })
```

Abbildung 12: User Database Struktur

B.7 API und Database Migration

Durchführung OOP-Setups in der Hauptklasse MyApplication:

```
63     def configure_app(self):
64         self.SECRET_KEY = self.app.config['SECRET_KEY']
65         self.app.config.from_object(self.config_class)
66         self.app.secret_key = self.app.config.get('SECRET_KEY', self.SECRET_KEY)
67
68     def setup_api(self):
69         self.api = Api(self.app, version='1.0', title='IAD odoo API', doc="/home",
70                        authorizations={
71                            'apiKey': {
72                                'type': 'apiKey',
73                                'in': 'header',
74                                'name': 'Authorization',
75                            },
76                            'basicAuth': {
77                                'type': 'basic',
78                            }
79                        },
80                        security=['BearerAuth', 'basicAuth'])
81
82     def setup_db(self):
83         migrate = Migrate(self.app, self.db)
84         if os.environ.get('FLASK_ENV') != 'testing':
85             database_file = 'iad_database2.db'
86             engine = create_engine(f'sqlite:///instance/{database_file}', echo=False)
87             self.db.init_app(self.app)
88             Migrate(self.app, self.db)
89             with self.app.app_context():
90                 self.db.create_all()
91                 migrate.init_app(self.app, self.db)
92                 self.Base.metadata.create_all(bind=engine)
93                 self.db.session.commit()
94                 self.Base.metadata.create_all(bind=engine)
95         else:
```

Abbildung 13: DB Migration Und API Einrichtung)

B.8 Odoo Endpunkte und Ressourcen

Durchführung Odoo endpunkte und Dokumentation des Quell Code, zusätzlich Fehler Protokollierung.

```
1  def setup_odoo_routes(ns_odoo, app, auth, instance_get_odoo_data,
2      odoo_model_measur_number, odoo_model_all,
3          odoo_model_measure_name, odoo_model_partner,
4          odoo_model_special, odoo_model_measur):
5
6      logging.basicConfig(level=logging.DEBUG, format='%(asctime)s - %(levelname)s
7          - %(message)s')
8
9      class CustomError(Exception):
10         def __init__(self, message, status_code):
11             Exception.__init__(self)
12             self.message = message
13             self.status_code = status_code
14
15     @app.errorhandler(CustomError)
16     def handle_custom_error(error):
17         logging.error(f'Benutzerdefinierter Fehler: {error.message}')
18         return {'message': error.message, error.status_code
19
20     """
21     *****eine kurze erkläerung fuer die oben genante parameter*****
22     : param ns_odoo: namespace nur fuer die odoo routes.
23     : param app: application instance.
24     : param auth: fuer die sicherheit massnahmen anmeldung.
25     : param instance_get_odoo_data: odoo connector class. damit kann man alle
26         daten von odoo kriegen.
27     : param odoo_model_measur_number: eine DB model struktur damit man
28         Informationen anhang des Number findet.
29     : param odoo_model_all: eine limitiertes route fuer die zukunft. um bestimmte
30         daten von
31         mehrere models zusammen in ein form zu haben.
32     : param odoo_model_measure_name: daten mit aus model measure kriegen durch
33         den massnahmen name.
34     : param odoo_model_partner: limitierte daten aus database model partner.
35     : param odoo_model_special: limitierte daten aus databaes model special days.
36     : param odoo_model_measur: limitierte daten aus database model massnahmen.
37     """
38
39     parser_string = reqparse.RequestParser()
40     parser_string.add_argument('number', type=str, help='Number must be an String
41         ', location='args')
42
43     parser_integer = reqparse.RequestParser()
44     parser_integer.add_argument('number', type=str, help='Number must be an
45         Integer', location='args')
46
47     @ns_odoo.route('/Get_data_by_measur_number/<string:number>')
48     class OdooDataByNumber(Resource):
49         @auth.login_required
50         @ns_odoo.doc(description="just authenticated users have access to this
51             data", security='basicAuth')
52         @ns_odoo.marshal_list_with(odoo_model_measur_number) # Optional field fuer
53             schema
```

Abbildung 14: Fehler-Protokollierung und Quell Code-Dokumentation

B.9 Tests und Modulare struktur des Projektes

Erfolgreiche Durchführung von Unittests und Integrationstests neben einer Darstellung der modularen Struktur des Projekts.

The screenshot displays a code editor with a project structure on the left and test execution results at the bottom. The project structure includes folders like `models`, `odoo_connector`, `resources`, `templates`, `utils`, `instance`, `tests`, and `venv`. The `tests` folder contains `pytest` and `unit` subfolders. The `unit` folder contains `test_odoo_models.py`, which is the active file. The code in `test_odoo_models.py` includes tests for `odoo_model_partner`, `odoo_model_special`, and `odoo_model_all`. The test execution results at the bottom show that 6 tests passed in 14 ms. The test results are summarized in a table:

Test Name	Duration
test_odoo_models	14 ms
OdooModelsTestCase	14 ms
test_odoo_model_partner	4 ms
test_create_odoo_models	2 ms
test_odoo_model_measur	2 ms
test_odoo_model_measur_n	2 ms
test_odoo_model_measure_2	2 ms
test_odoo_model_special	2 ms

The test execution results also show a summary of the test run: 6 passed, 2 warnings in 1.03s. The test level percentage is 100%.

Abbildung 15: Test-Durchführung

C Anlage IHK

C.1 IHK Anlage 1

 IHK Industrie- und Handelskammer Erfurt	Anlage 1	
Bestätigung über die durchgeführte betriebliche Projektarbeit		
Bitte füllen Sie die Anlagen 1 bis 4 in Druckschrift oder maschinell aus und fügen Sie diese der Projektdokumentation bei!		
Antragsteller: Name: Vorname: Straße: PLZ Ort: EMail:	Ausbildungsbetrieb (Praktikumsbetrieb): IAD GmbH	
Projektbezeichnung (Auftrag/Teilauftrag): Unternehmensressourcenplanung (Middleware Secure API zur Pflege und Abfrage Daten von Odoo)		
19.02.2024	04.03.2024	80 std
Durchführungsbeginn	Durchführungsende	Zeitaufwand in Stunden
Bestätigung des Ausbildungsbetriebes/Praktikumsbetriebes: Wir versichern, dass das Projekt, wie in der Dokumentation dargestellt, in unserem Unternehmen realisiert wurde.		
Name, Vorname (Projektverantwortlicher)		Stempel u. Unterschrift (Unternehmen)
Verbindliche Erklärung des Prüfungsteilnehmers/der Prüfungsteilnehmerin: Ich versichere, durch meine Unterschrift, dass ich das Projekt und die dazugehörige Dokumentation selbständig und ohne fremde Hilfe angefertigt und alle Stellen, die ich wörtlich oder annähernd wörtlich aus Veröffentlichungen entnommen habe, als solche kenntlich gemacht habe. Die Arbeit hat in dieser Form keiner anderen Prüfungsinstitution vorgelegen.		
Erfurt, 02.04.2024		Unterschrift Prüfungsteilnehmer
Ort, Datum		

D116 V: 1.2

1/4

Abbildung 19: IHK Anlage 1


IHK

Industrie- und Handelskammer
Erfurt

Anlage 2

Protokoll über die betriebliche Projektarbeit (vom Projektverantwortlichen im Unternehmen auszufüllen)

1. Arbeitszeit

- Die vom Prüfungsteilnehmer kalkulierte Zeit entspricht der betrieblichen Kalkulation
☒ ja ☐ nein

Wenn nein: Sie ist um% höher, um% niedriger

- Das Projekt wurde vom Prüfungsteilnehmer in der kalkulierten Zeit komplett fertiggestellt (einschließlich eventueller Nacharbeit):
☒ ja ☐ nein

Wenn nein: Um Std. früher fertig geworden
..... Std. länger gebraucht

2. Ausführung

- Wurde das Projekt entsprechend dem eingereichten Konzept ausgeführt?
☒ ja ☐ nein

Wenn nein: Begründung und Umfang der Hilfestellung:
.....
.....

- Das Projekt konnte ohne Nacharbeit im einwandfreien Zustand übergeben werden
☒ ja ☐ nein

Wenn nein: Begründung:
.....
Umfang der Nacharbeit:
.....
.....

3. Dokumentation

- Die Dokumentation wurde vom Prüfungsteilnehmer selbständig ohne fremde Hilfe erstellt.
☒ ja ☐ nein

Wenn nein: Worin bestehen die Abweichungen?
.....
.....

Wir bestätigen, dass die obigen Angaben wahrheitsgemäß sind.

2.4.24

Datum

Unterschrift
Projektverantwortlicher

Unterschrift
Prüfungsteilnehmer

D116 V: 1.2

2/4

C.3 IHK Anlage 3



Industrie- und Handelskammer
Erfurt

Anlage 3

Projekttagbuch

Datum	Tätigkeiten	Besonderheiten
19.02.2024	Durchführung der Ist-Analyse Durchführung der Soll-Analyse Durchführung der Wirtschaftlichkeitsanalyse Erstellung des Lastenheftes	
20.02.2024	Zielform festlegen-/Architekturentwurf	
20.02.2024	Funktionsplanung	
20.02.2024	Kosten/Nutzenanalyse	
21.02.2024	Pflichtenheft erstellen	
21.02.2024	Entwicklungsumgebung einrichten	
21.02.2024	Anlegen der Datenbank und Datenbankmigration ¹	1- Datenbank Migration und Implementierung Wegen ein Konflikt hatte es 2 stunden länger gedauert als ich gedacht hatte, so habe ich von Dokumentation zeit,2 std. minimiert um das Problem zu beheben. deswegen bei der Dokumentation bleibt 18 stunden.
21.02.2024 -22.02.2024	Implementierung der Datenbankstruktur und benutzer Rechten ¹	
22.02.2024	Implementierung der API mit Flask Rest-x	
22.02.2024 -23.02.2024	Implementierung der Benutzer Authorisierung und Authentifizierung Logik	
23.02.2024	Implementierung der Odoo Integration und connector ²	
23.02.2024 -26.02.2024	Implementierung des Odoo Route-/Endpunkten	
26.02.2024	Implementierung des User Route-/ Endpunkte	
26.02.2024	Einrichtung des Sicherheitsmaßnahmen	
26.02.2024	Implementierung des Mail Services und support routes	
27.02.2024	Implementierung des Datenbank schema-/Model	
27.02.2024	Durchführung und Implementierung des Einheiten Test	2- Kleine Konflikte während der Implementierung von Unit-Tests mit Odoo: Die Unit-Tests konnten nicht wie gewünscht durchgeführt werden, da ich eine Stunde benötigte, um den Fehler zu automatisieren und zu beheben."
27.02.2024 -28.02.2024	Durchführung und Implementierung des Integrations Test	
28.02.2024	durchführung System Test	
28.02.2024	Fehlerbehebung und Identifizierung	
28.02.2024	Abnahme durch Die Fachabteilung	
28.02.2024 -29.02.2024	Erstellen der Projektdokumentation	
29.02.2024 -01.03.2024	Erstellen der Entwicklerdokumentation	
01.03.2024 -04.03.2024	Erstellen der Benutzerdokumentation	

D116 V: 1.2

3/4

Abbildung 21: IHK Anlage 3

C.4 IHK Anlage 4



Industrie- und Handelskammer
Erfurt

Anlage 4

Zeitaufwand für Prozessschritte

Durchführungsort	Tätigkeiten	benötigte Zeit in Std.
Das Ganze Projekt wurde bei der IAD GmbH durch geführt.	Analysephase:	8 std.
	- Durchführung der Ist-Analyse	2
	- Durchführung der Soll-Analyse	2
	- Durchführung der Wirtschaftlichkeitsanalyse	2
	- Erstellung des Lastenheftes	2
	Planungsphase:	10 std.
	- Zielplattform festlegen-/Architekturentwurf	3
	- Funktionsplanung	2
	- Funktionsplanung	3
	- Pflichtenheft erstellen	2
	Durchführung:	32 std.
	1- Entwicklungsumgebung einrichten.	2
	2- Anlegen der Datenbank und Datenbankmigration.	2
	3- Implementierung der Datenbankstruktur und Benutzer Rechten.	5
	4- Implementierung der API mit Flask Rest-x.	3
	5- Implementierung der Benutzer Authorisierung und Authentifizierung Logik.	3
	- Implementierung der Odoo Integration und connector.	3
	- Implementierung des Odoo Route-/Endpunkten.	5
	- Implementierung des User Route-/ Endpunkte.	3
	- Einrichtung des Sicherheitsmaßnahmen.	2
	- Implementierung des Mail Services und support routes.	2
	- Implementierung des Datenbank schema-/Model.	2
	Testphase:	12 std.
	- Durchführung und Implementierung des Einheiten Test.	5
	- Durchführung und Implementierung des Integrations Test.	2
	- Durchführung System Test	2
	- Fehlerbehebung und Identifizierung	2
	- Abnahme durch Die Fachabteilung	1
	Dokumentation:	18 std.
	- Erstellen der Projektdokumentation	10
	- Erstellen der Entwicklerdokumentation	3
	- Erstellen der Benutzerdokumentation	5
Summe		62 std.
Dokumentation der Tätigkeiten		18 std.
Gesamtzeit		80 std.